

CONSOLATRIX COLLEGE OF TOLEDO CITY INC.

COLLEGE OF COMPUTER STUDIES

Software Requirements Specifications

for

HotelEase: BSHM Property Management System

Change History

Table of Contents

1. Introduction	4
1.1. Purpose	4
1.2. Scope	4
1.3. Definitions, Acronyms and Abbreviations	5
1.4. References	6
2. Overall Description	7
2.1. Product perspective	7
2.2. User characteristics	9
• Guest	9
• Front Office (FO)	9
• Admin	9
o Role: System administrator with full access to configuration and analytics.	9
o Privileges:	9
▪ Access the Analytics Dashboard (occupancy, revenue, trends, peaks).	9
▪ Manage all user accounts: view, assign roles (guest/fo/admin), delete.	9
▪ Full room management: add, edit, activate/deactivate rooms with photos.	9
▪ Configure system settings.	9
▪ Enable/disable Training Mode for classroom use.	9
▪ Generate time-limited training session codes.	9
▪ Reset all training/demo data across sandboxed collections.	9
2.4. Constraints	10
2.5. Assumptions and dependencies	11
3. Specific Requirements	12
3.1. External interface requirements	12
3.1.1. Hardware interfaces	12
3.1.2. Software interfaces	12
3.1.3. Communications interfaces	13
3.2. Functional requirements	14
Module 1 – User & Authentication Management	14
Module 2 – Room & Booking Management	22
Module 3 – Housekeeping, Payments & Operations	29
Module 4 – Administration & Analytics	41
3.4 Non-functional requirements	45
Performance	45
Security	45
Reliability	46

1. Introduction

1.1. Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a comprehensive and detailed description of the HotelEase system — a web-based Property Management System (PMS) developed for the Bachelor of Science in Hospitality Management (BSHM) department. This document defines the functional and non-functional requirements necessary for the successful development, deployment, and maintenance of the software, ensuring a shared understanding among developers, testers, stakeholders, and academic advisors.

1.2. Scope

HotelEase is a web-based hotel/property management system that simulates a real-world hotel environment to support academic learning, departmental operations, and practical training for BSHM students. The system mirrors industry-standard PMS workflows while being tailored to the academic context of the institution.

- Key features of the platform include:
 - Room Management: Administrators can add, edit, activate, and deactivate rooms with details such as room type, rate, amenities, floor, and photo gallery.
 - Online Booking: Guests can browse available rooms, select dates, and submit booking requests. The system includes double-booking prevention with date-overlap validation.
 - Booking Lifecycle: Full workflow from Pending → Approved → Checked In → Checked Out → Cancelled, with Front Office (FO) staff managing approvals, check-ins, and check-outs.
 - Housekeeping Workflow: Automated room status transitions (Dirty → Being Cleaned → Pending Approval → Available) with audit logging of all status changes.
 - Payment Processing: Support for GCash, Cash, Check, and Credit Card payments with partial payment/deposit tracking, folio management, and PDF receipt generation via jsPDF.
 - Announcements & Events: FO staff can create, edit, and publish announcements with optional image uploads via Cloudinary, visible on the guest-facing landing page.
 - Guest Reviews & Testimonials: Guests can submit star-rated reviews for rooms and general testimonials, with FO moderation (approve/reject) for testimonials.
 - AI Chatbot: A Groq-powered (LLaMA 3.1 8B) chatbot widget on the landing page that answers hotel-related inquiries using real-time room data.
 - In-App Notifications: Real-time notification system for booking events, housekeeping alerts, announcements, and support messages across all user roles.
 - Contact & Support Messaging: Guests can submit support messages; FO staff can view, mark as read, and reply with optional email notification via Firebase Cloud Functions.
 - Analytics Dashboard: Admin dashboard showing occupancy rates, revenue summaries, booking trends (7-day chart), and peak booking periods.
 - Training/Demo Mode: Admin-controlled training mode with session codes, sandboxed Firestore collections (training_ prefix), and one-click data reset for classroom use.
- Role-Based Access Control: Three distinct roles (Guest, Front Office, Admin) with Firestore security rules and client-side route protection via PrivateRoute component.

The system is not designed for comprehensive learning management (LMS) functionalities such as grading, assignment tracking, or curriculum management. It does not process real financial transactions or integrate

with payment gateways for live monetary transfers.

Application Benefits:

- BSHM Students: Gain hands-on experience with industry-standard hotel management workflows in a safe, sandboxed training environment.
- Faculty/Instructors: Facilitate practical exercises using the training mode with session codes and resettable demo data.
- Department: Streamline room booking operations, guest management, and event announcements for departmental activities.
- Institution: Serve as a capstone project demonstrating full-stack web development competency with real-world applicability.

The objectives of **HotelEase** are to:

- Provide a centralized, web-accessible platform for managing hotel room bookings, guest check-ins/check-outs, payments, and housekeeping operations.
- Implement a role-based access control system with three distinct user types (Guest, Front Office, Admin) enforced at both client and database levels.
- Deliver a training/demo mode that allows instructors to create sandboxed sessions for student learning without affecting production data.
- Integrate an AI-powered chatbot to assist guests with hotel-related inquiries using real-time room data.
- Generate professional PDF receipts and provide analytics dashboards for operational insights.

1.3. Definitions, Acronyms and Abbreviations

Term	Definition
HotelEase	The name of the web-based Property Management System for the BSHM department.
PMS	Property Management System — software used by hotels to manage front-office operations.
Guest	A registered user who can browse rooms, make bookings, submit reviews, and interact with the chatbot.
Front Office (FO)	A staff user responsible for managing check-ins, check-outs, payments, housekeeping, announcements, and guest communications.
Admin	A privileged user who manages system settings, user roles, room inventory, analytics, and the training mode.

Firestore	Google Cloud Firestore — the NoSQL cloud database used for all data persistence in HotelEase.
Firebase Auth	Firebase Authentication service used for user registration, login, and anonymous sign-in for training mode.
Training Mode	A sandboxed operational mode where all data operations are routed to separate Firestore collections prefixed with 'training_'.
Cloudinary	A cloud-based image management service used for uploading and hosting room photos and announcement images.
Groq	An AI inference platform used to power the HotelEase chatbot via the LLaMA 3.1 8B model.
Folio	An itemized billing summary for a guest's stay, tracking total cost, deposits, and outstanding balance.
RBAC	Role-Based Access Control — a security mechanism restricting system access based on assigned user roles.

1.4. References

- IEEE Std 830-1998: IEEE Standard for Software Requirements Specifications

Date: 1998 | Publisher: Institute of Electrical and Electronics Engineers (IEEE) | Source: IEEE Xplore Digital Library

- Firebase Documentation: Cloud Firestore, Firebase Auth, Cloud Functions

Date: Ongoing | Publisher: Google LLC | Source: <https://firebase.google.com/docs>

- React Documentation: React 19 Official Documentation

Date: Ongoing | Publisher: Meta Platforms, Inc. | Source: <https://react.dev>

- Vite Documentation: Vite Build Tool Documentation

Date: Ongoing | Publisher: Evan You / Vite Contributors | Source: <https://vitejs.dev>

- Groq API Documentation: Groq Cloud AI Inference Platform

Date: Ongoing | Publisher: Groq, Inc. | Source: <https://console.groq.com/docs>

- Cloudinary Documentation: Image and Video Management API

Date: Ongoing | Publisher: Cloudinary Ltd. | Source: <https://cloudinary.com/documentation>

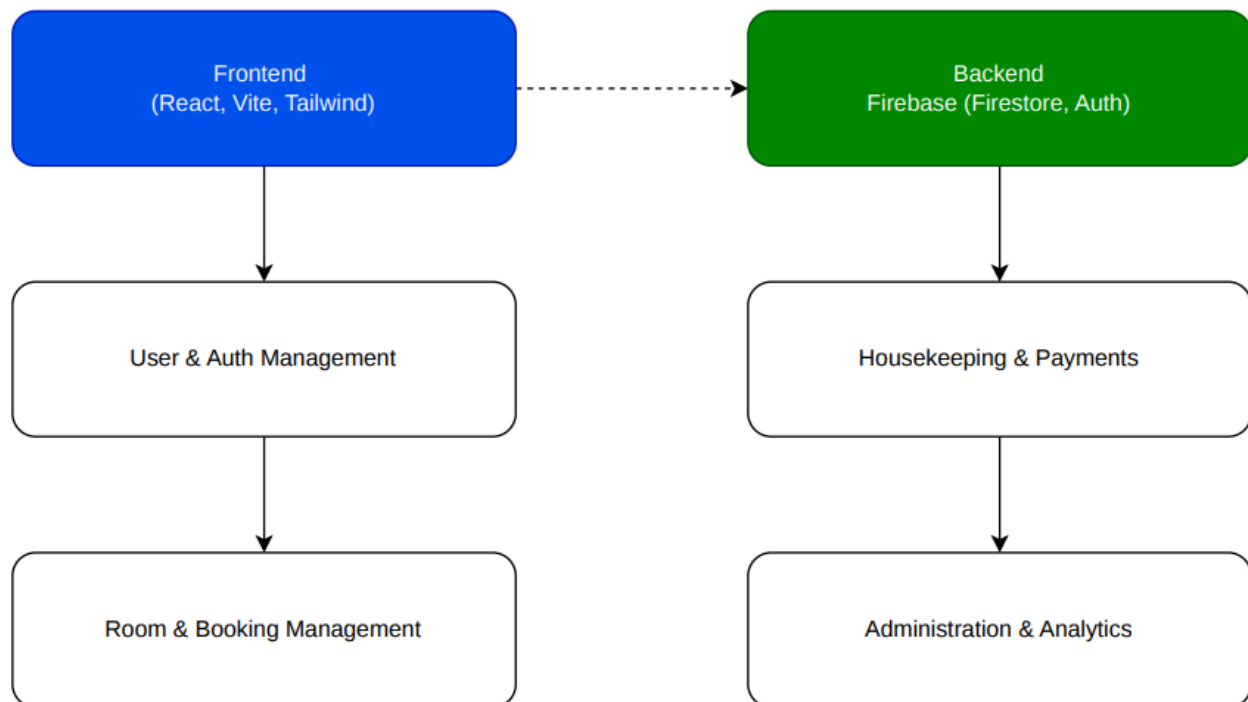
2. Overall Description

2.1. Product perspective

HotelEase is a self-contained, web-based Property Management System built with a modern JavaScript stack. The frontend is a React 19 single-page application bundled with Vite 8, styled with Tailwind CSS 3.4, and enhanced with shadcn/ui components (Radix UI primitives). The backend relies entirely on Firebase services: Cloud Firestore for data persistence, Firebase Authentication for identity management, and Firebase Cloud Functions for server-side operations like email notifications. Image assets (room photos, announcement images) are stored on Cloudinary CDN via unsigned uploads. The AI chatbot is powered by the Groq inference platform using the LLaMA 3.1 8B model.

The system operates independently and does not require integration with external hotel management systems. However, its modular service-layer architecture (with dedicated service files for each domain) allows for future expansion, such as integrating payment gateways or external booking aggregators.

A key architectural feature is the Training Mode, which routes all data operations to sandboxed Firestore collections (prefixed with 'training_') via a centralized utility function (getCol in db-utils.js). This ensures complete isolation between production and training data.



Modular Decomposition

Module 1: User & Authentication Management

- Transaction 1.1: Guest Registration (email/password via Firebase Auth)
- Transaction 1.2: User Login (email/password authentication)
- Transaction 1.3: Password Reset (Firebase sendPasswordResetEmail)
- Transaction 1.4: Anonymous Sign-In for Training Mode
- Transaction 1.5: User Role Management (Admin assigns guest/fo/admin roles)
- Transaction 1.6: User Profile Management (view/edit profile, delete user)

Module 2: Room & Booking Management

- Transaction 2.1: Browse and Filter Available Rooms
- Transaction 2.2: View Room Details, Photos, Amenities, and Reviews
- Transaction 2.3: Create Online Booking (with date validation and conflict detection)
- Transaction 2.4: Approve or Reject Booking Requests (FO)
- Transaction 2.5: Guest Check-In (FO — triggers room status to Occupied)
- Transaction 2.6: Guest Check-Out (FO — triggers room status to Dirty)
- Transaction 2.7: Cancel Booking (Guest or FO)
- Transaction 2.8: View Booking History and Status (Guest: My Bookings)
- Transaction 2.9: Room CRUD — Add, Edit, Activate, Deactivate Rooms (Admin)
- Transaction 2.10: Adjust Room Rates (FO)

Module 3: Housekeeping, Payments & Operations

- Transaction 3.1: Update Room Cleaning Status (Dirty → Being Cleaned → Pending Approval → Available)
- Transaction 3.2: Record Payment (GCash, Cash, Check, Credit Card with partial deposit tracking)
- Transaction 3.3: Generate and Download PDF Receipt (jsPDF + AutoTable)
- Transaction 3.4: View Payment History for a Booking
- Transaction 3.5: Create, Edit, and Delete Announcements (with Cloudinary image upload)
- Transaction 3.6: Submit and Moderate Guest Testimonials (Pending → Approved/Rejected)
- Transaction 3.7: Submit Room Reviews (star rating + feedback after checkout)
- Transaction 3.8: Guest Contact/Support Messaging with FO Reply

Module 4: Administration & Analytics

- Transaction 4.1: Analytics Dashboard (occupancy rate, revenue, booking trends, peak days)
- Transaction 4.2: User Management (view all users, change roles, delete accounts)
- Transaction 4.3: System Settings Configuration
- Transaction 4.4: Toggle Training Mode ON/OFF
- Transaction 4.5: Generate Training Session Code (with TTL expiry)
- Transaction 4.6: Reset All Training Data (clears sandboxed collections)

2.2. User characteristics

HotelEase is designed for three types of users with specific roles and privileges:

- **Guest**
 - **Role: Primary end-user of the system.**
 - Privileges:
 - · Register an account and log in with email/password.
 - · Browse available rooms with details, photos, amenities, and rates.
 - · Create online booking requests with date selection and conflict validation.
 - · View booking history and status updates (My Bookings page).
 - · Cancel pending or approved bookings.
 - · Submit star-rated reviews for rooms after checkout.
 - · Submit general testimonials (subject to FO moderation).
 - · Interact with the AI chatbot for hotel inquiries.
 - · Receive in-app notifications for booking status changes and announcements.
 - · Send contact/support messages to Front Office staff.
 - · View and manage personal profile.
- **Front Office (FO)**
 - **Role: Operational staff managing day-to-day hotel front desk functions.**
 - Privileges:
 - · Access the FO Dashboard with color-coded room status grid.
 - · Approve or reject guest booking requests with notification to guests.
 - · Process guest check-in (booking → Checked In, room → Occupied).
 - · Process guest check-out (booking → Checked Out, room → Dirty).
 - · Manage housekeeping workflow (update room cleaning status with audit logging).
 - · Record payments (GCash, Cash, Check, Credit Card) with folio tracking.
 - · Generate and download professional PDF receipts.
 - · Create, edit, and delete announcements with optional image uploads.
 - · Adjust room rates per room.
 - · View and manage all bookings across the system.
 - · Moderate guest testimonials (approve/reject).
 - · Read and reply to guest support messages.
 - · Receive in-app notifications for new bookings, cancellations, and dirty rooms.
 - ·
- **Admin**
 - **Role: System administrator with full access to configuration and analytics.**
 - Privileges:
 - · Access the Analytics Dashboard (occupancy, revenue, trends, peaks).
 - · Manage all user accounts: view, assign roles (guest/fo/admin), delete.
 - · Full room management: add, edit, activate/deactivate rooms with photos.
 - · Configure system settings.
 - · Enable/disable Training Mode for classroom use.
 - · Generate time-limited training session codes.
 - · Reset all training/demo data across sandboxed collections.

2.4. Constraints

Several constraints govern the development and operational scope of HotelEase:

Regulatory Policies:

- The system must comply with the Philippine Data Privacy Act of 2012 (RA 10173) for handling personal guest data including names, emails, and booking information.

Hardware Limitations:

- The system is web-based and runs on standard web infrastructure. No specialized hardware is required, though server performance may be affected by concurrent Firestore read/write operations during peak usage.

Interfaces to Other Applications:

- The current version integrates with Firebase (Auth, Firestore, Cloud Functions), Cloudinary (image hosting), and Groq (AI chatbot). Future versions may integrate with payment gateways or external booking platforms.

Parallel Operation:

- The system supports multiple simultaneous users across all roles. Firestore transactions (runTransaction) are used for critical operations like booking creation, check-in/check-out, and payment recording to prevent race conditions.

Audit Functions:

- All housekeeping status changes are logged in the housekeeping_logs collection with timestamps, from/to status, and the role of the user who made the change.

Reliability Requirements:

- The system targets high availability by leveraging Firebase's managed infrastructure. Client-side error handling with graceful fallbacks ensures users are notified of failures.

Criticality of the Application:

- Medium criticality — the platform supports academic operations and training. While not life-critical, extended downtime would disrupt scheduled classroom exercises and departmental booking operations.

Safety and Security Considerations:

- Firebase Auth provides encrypted credential management. Firestore Security Rules enforce role-based access at the database level. HTTPS is enforced for all communications. Anonymous users (training mode) have restricted write access. Input validation is applied on both client and database rule levels.

2.5. Assumptions and dependencies

The following assumptions were made during design and development:

- The application will be deployed using Firebase Hosting (frontend) and Firebase Cloud Functions (backend APIs and email triggers).
- Users will access the platform through modern web browsers supporting HTML5, CSS3, ES2020+ JavaScript, and WebSocket connections for Firestore real-time listeners.
- A stable internet connection is required for all operations, as the system relies entirely on cloud services (Firebase, Cloudinary, Groq).
- Environment variables (API keys for Firebase, Cloudinary, Groq) will be properly configured in the deployment environment.
- The Groq API service will remain available for chatbot functionality; if unavailable, the chatbot gracefully returns a fallback message.
- Cloudinary's free tier will be sufficient for the expected volume of image uploads during academic use.
- Training mode sessions will be managed by instructors who generate session codes and reset data between class sessions.

3. Specific Requirements

3.1. External interface requirements

3.1.1. Hardware interfaces

HotelEase is a cloud-hosted web application with minimal local hardware dependencies. All server-side operations are managed by Firebase's infrastructure.

Cloud Infrastructure:

- Cloud Infrastructure: Google Cloud Platform (via Firebase) handles all server-side compute, storage, and networking. No on-premise server hardware is required.
- Firestore Database: Managed by Google Cloud with automatic scaling, replication, and backup.
- Cloud Functions: Serverless compute for email notifications and backend logic.

Supported Devices:

- Desktop and laptop computers: Full-screen responsive interface optimized for 1024px+ viewport widths.
- Tablets: Responsive layout adapts to medium-sized screens (768px–1024px).
- Mobile devices (smartphones): Responsive mobile layout with collapsible sidebar navigation and touch-friendly UI elements.

3.1.2. Software interfaces

The system integrates several software components and third-party services:

Frontend Stack:

- React 19 — Component-based UI library for building the single-page application.
- Vite 8 — Next-generation frontend build tool and development server.
- Tailwind CSS 3.4 — Utility-first CSS framework for responsive styling.
- shadcn/ui (Radix UI) — Accessible, unstyled UI component primitives.
- React Router v7 — Client-side routing with role-based route protection (PrivateRoute).
- FullCalendar.js — Interactive booking calendar component.
- Framer Motion — Animation library for UI transitions and micro-interactions.
- Recharts — Charting library for analytics dashboard visualizations.
- jsPDF + jsPDF-AutoTable — Client-side PDF receipt generation.
- Lucide React — Icon library for consistent UI iconography.

Backend / Cloud Services:

- Firebase Authentication — Email/password and anonymous sign-in for user identity management.
- Cloud Firestore — NoSQL document database for all persistent data storage.
- Firebase Cloud Functions — Serverless functions for email triggers (Nodemailer + Gmail SMTP).
- Firebase Hosting — Static site hosting for the production frontend build.

Third-Party Integrations:

- • Cloudinary — Cloud-based image upload, transformation, and CDN delivery for room photos and announcement images. Uses unsigned upload presets.
- • Groq API — AI inference platform powering the chatbot via the LLaMA 3.1 8B Instant model. Context-aware with real-time room data injection

3.1.3. Communications interfaces

Network Protocols:

- • HTTPS is enforced for all client-server communication to protect sensitive data including credentials, booking details, and payment information.
- • WebSocket connections are used by Firestore's onSnapshot real-time listeners for live data updates across bookings, rooms, notifications, and messages.
- • TCP/IP is the underlying transport protocol for all network communication.

Email Services:

- • Firebase Cloud Functions integrate with Nodemailer using Gmail SMTP to send automated email notifications for booking confirmations, approvals, rejections, and support replies.
- • Support reply emails can be toggled on/off via the VITE_ENABLE_SUPPORT_REPLY_EMAIL environment variable.

API Endpoints:

- • Firebase Cloud Functions expose HTTPS callable endpoints for server-side operations such as sendSupportReplyEmail.
- • The Groq REST API is consumed directly from the client for chatbot inference (with the API key stored as an environment variable).
- • Cloudinary's Upload API is consumed directly from the client using unsigned upload presets for image management.

3.2. Functional requirements

Module 1 – User & Authentication Management

1.1 Transaction Name: Guest Registration

- Use Case Diagram

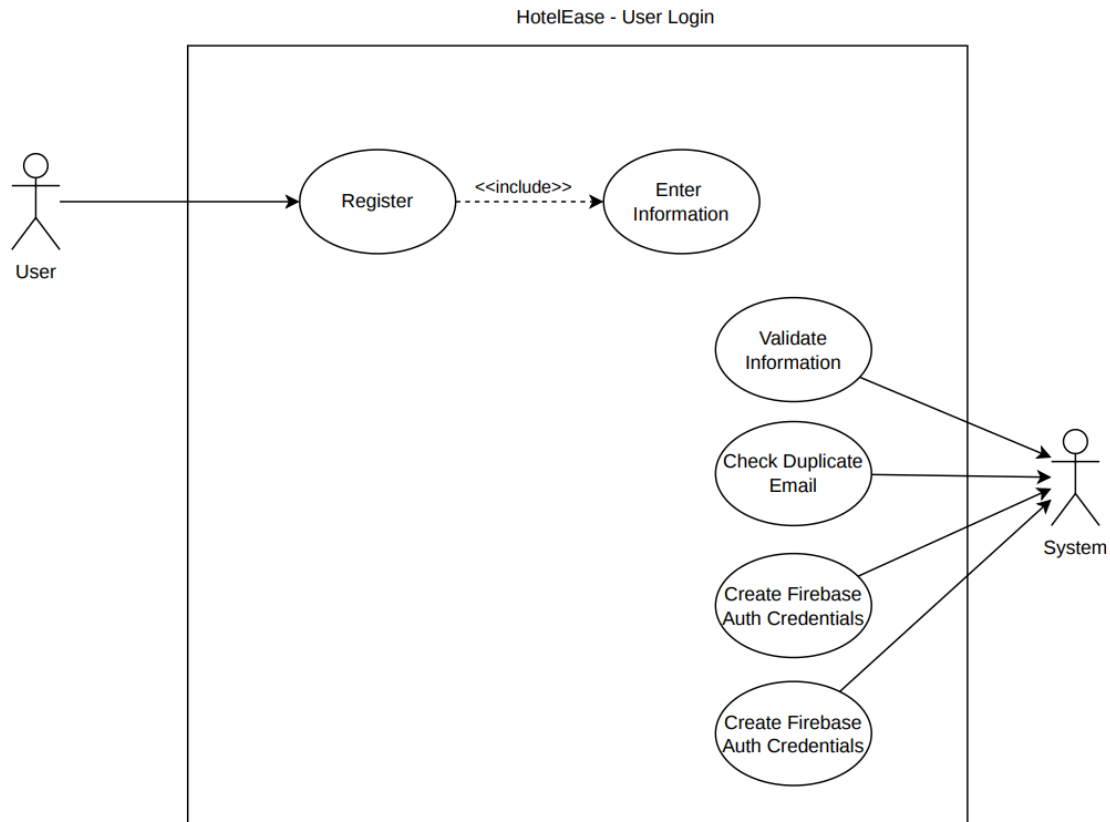


Figure 1. Use Case Diagram for User Registration

- **Use Case Description**

A new user provides their full name, email address, and password to create a guest account. The system creates a Firebase Auth credential and a corresponding user profile document in Firestore with the role set to 'guest'.

- · Preconditions: The user must not already have an account with the provided email. All required fields (full name, email, password) must be completed.
- · Postconditions: The user is registered in Firebase Auth and a profile document is created in the users collection. The user is automatically signed in and redirected to the landing page.

Main Success Scenario:

- 1. The guest navigates to the Registration page.
- 2. The guest enters their full name, email, and password.
- 3. The system validates the input fields.
- 4. Firebase Auth creates the user credential.
- 5. The system creates a user profile in Firestore with role 'guest'.
- 6. The guest is signed in and redirected to the landing page.

- Activity Diagram

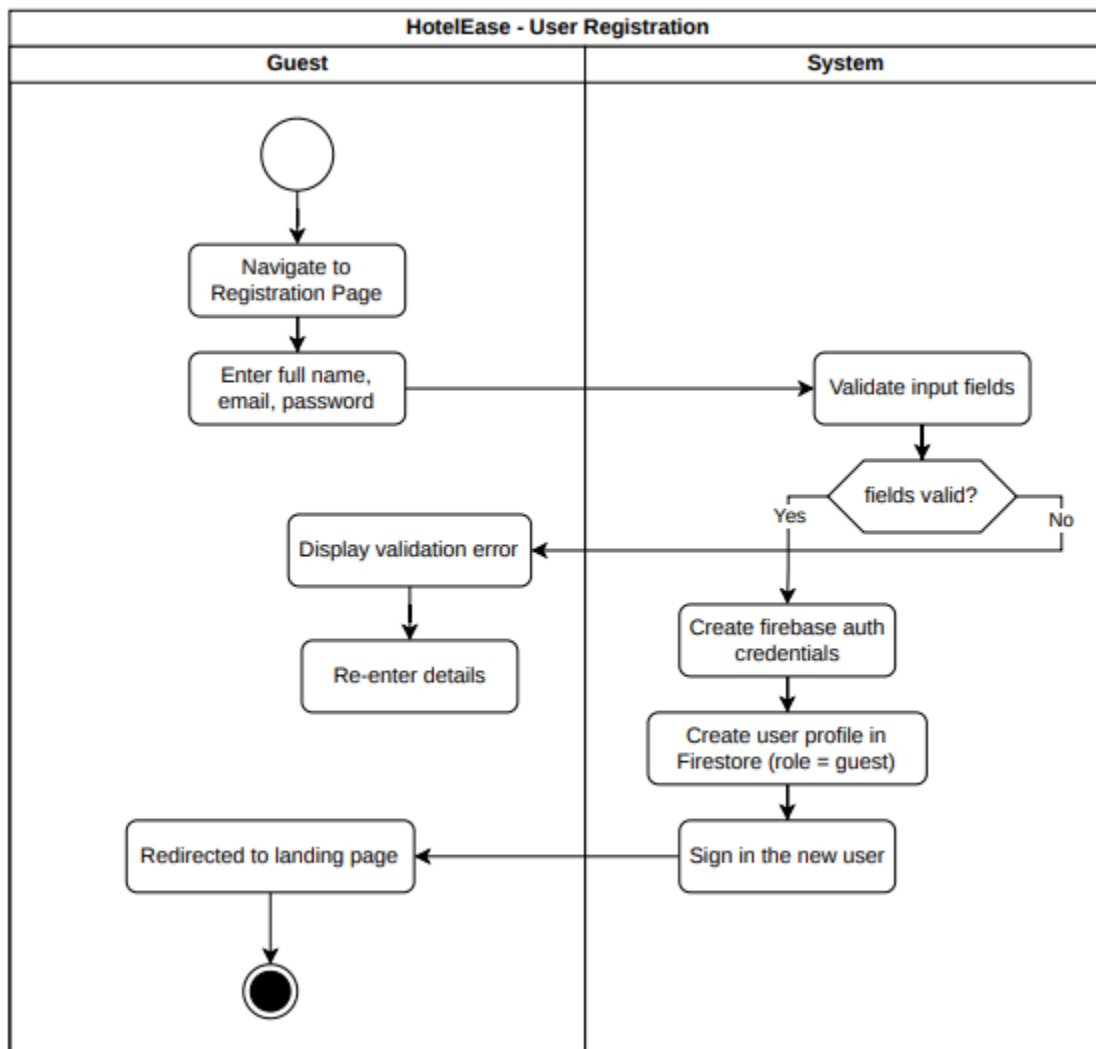


Figure 2. Activity Diagram for User Registration

- Wireframe

HotelEase BSHM-PMS Rooms Login Register

Join HotelEase

Create your account to start booking your luxury stay today.

Full Name
Juan Dela Cruz

Email
example@gmail.com

Password

Register

Already have an account? [Sign in here](#)

HotelEase © 2026 All rights reserved. About Contact Privacy

Figure 3. A User Registration Page

1.2 Transaction Name: User Login

- Use Case Diagram

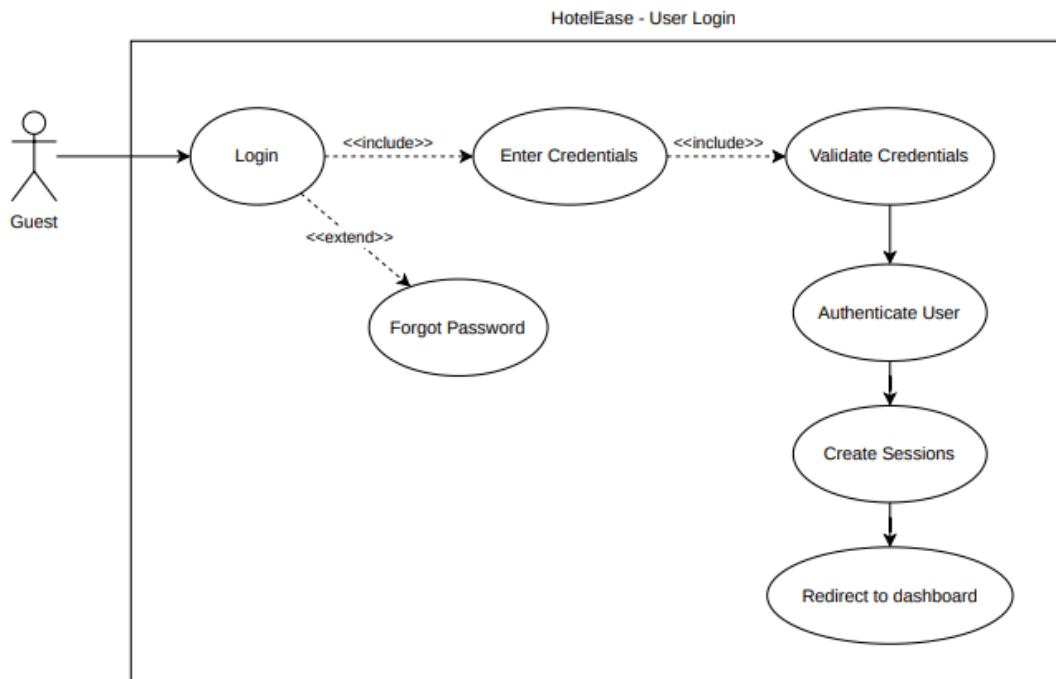


Figure 4. Use Case Diagram for User Login

- Use Case Description

- **Actors:** Registered User
- **Description:** The registered user provides their credentials (username and password) to log into the system. Upon validation, the user is granted access.
- **Preconditions:** The user must be registered, and the credentials must be correct.
- **Postconditions:** The user is logged in and directed to their dashboard.

- **Main Success Scenario:**

1. The user accesses the login form.
2. The user enters their credentials.
3. The system checks the credentials.
4. The user is logged in.

- Activity Diagram

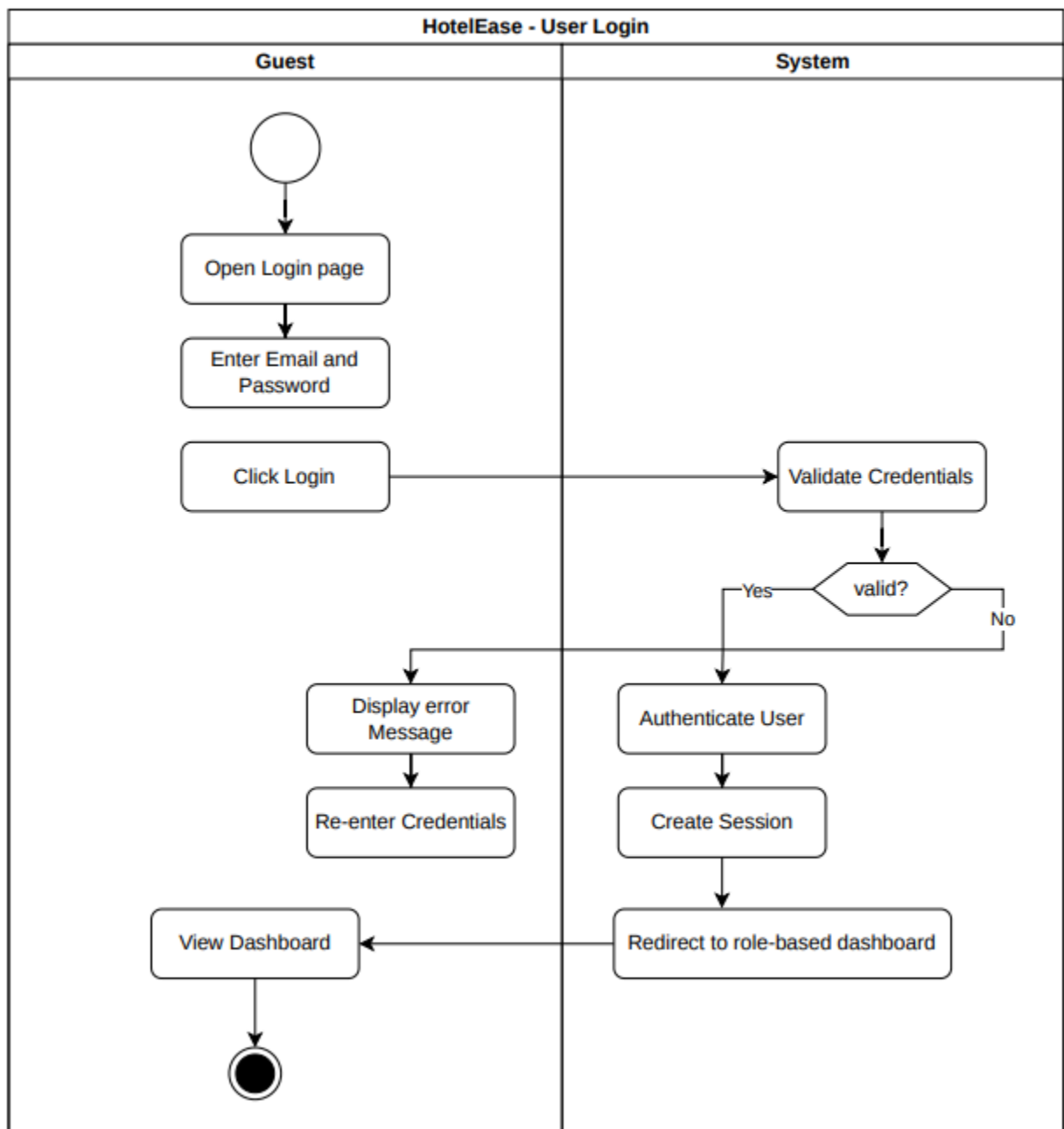
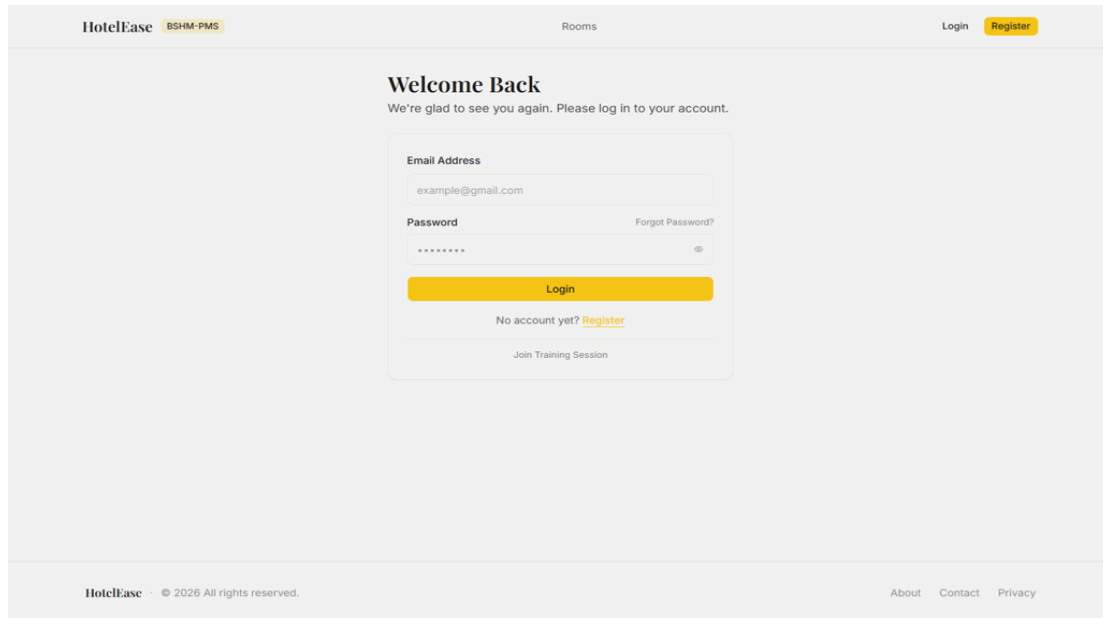


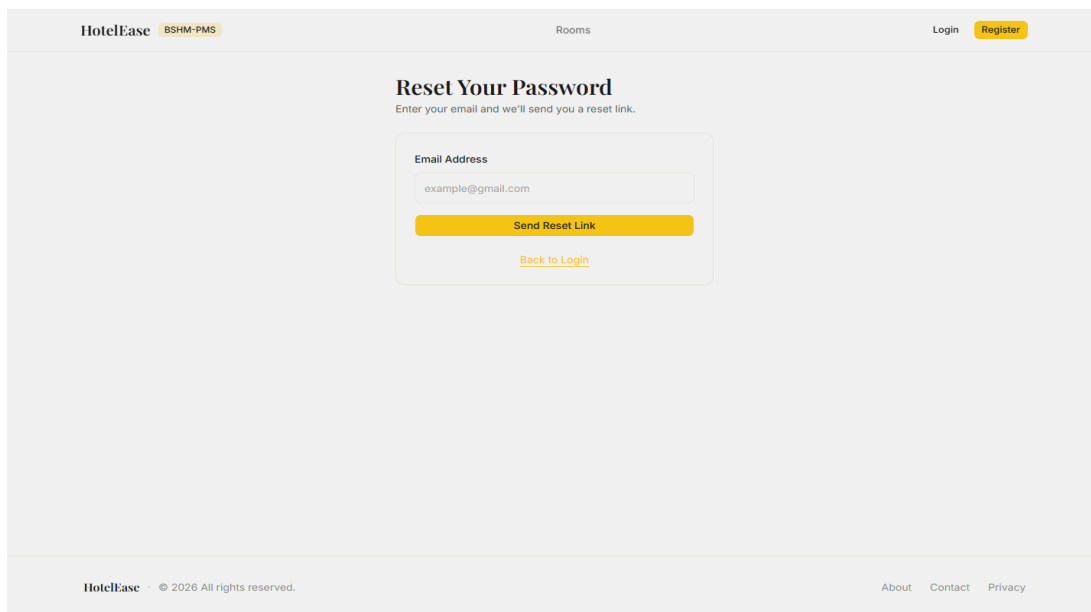
Figure 5. Activity Diagram for User Login

- **Wireframe**



The wireframe shows a login page for HotelEase. The header includes the logo 'HotelEase BSHM-PMS', a 'Rooms' link, and 'Login' and 'Register' buttons. The main content area features a 'Welcome Back' heading, a subheading 'We're glad to see you again. Please log in to your account.', and a login form. The form has an 'Email Address' field with 'example@gmail.com', a 'Password' field with masked characters and a toggle icon, a 'Forgot Password?' link, a yellow 'Login' button, a 'No account yet? Register' link, and a 'Join Training Session' link. The footer contains copyright information and links for 'About', 'Contact', and 'Privacy'.

Figure 6. A Login Page of HotelEase



The wireframe shows a reset password page for HotelEase. The header is identical to the login page. The main content area features a 'Reset Your Password' heading, a subheading 'Enter your email and we'll send you a reset link.', and a form. The form has an 'Email Address' field with 'example@gmail.com', a yellow 'Send Reset Link' button, and a 'Back to Login' link. The footer is identical to the login page.

Figure 7. A Reset Password Page of HotelEase

Module 2 – Room & Booking Management

2.1 Transaction Name: Online Room Booking

- Use Case Diagram

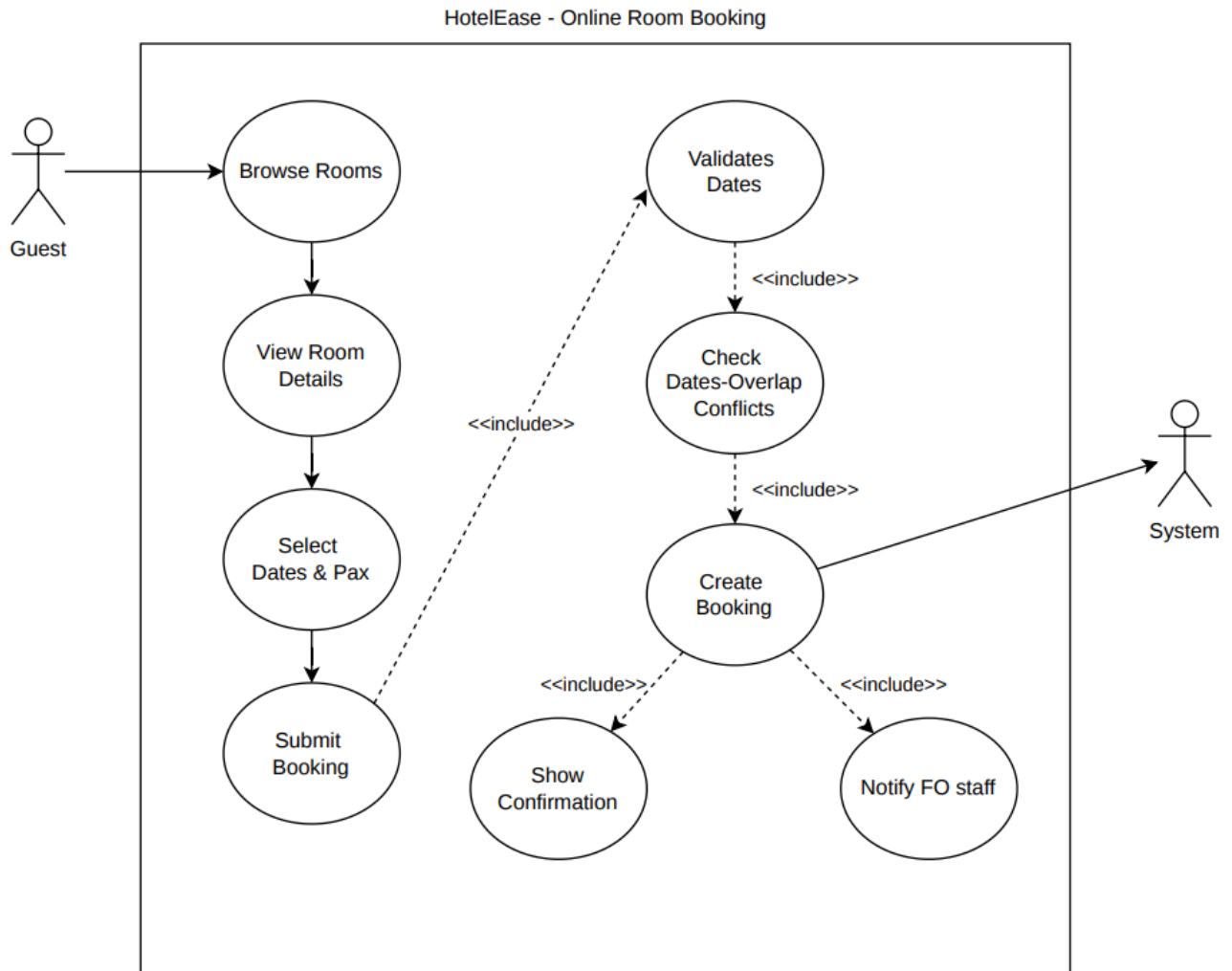


Figure 8. A Use Case Diagram for Room & Booking Management

- **Use Case Description**

An authenticated guest selects a room, chooses check-in and check-out dates, specifies the number of guests (pax) and any special requests, then submits the booking. The system validates dates, checks for conflicts against existing bookings, and creates the booking record in a Firestore transaction.

- · Preconditions: The guest must be logged in. The selected room must be active. Check-out date must be after check-in date. No overlapping bookings may exist for the same room and dates.
- · Postconditions: A new booking document is created with status 'Pending'. All FO staff receive an in-app notification about the new booking request. The total cost is calculated as (rate per night × number of nights).

Main Success Scenario:

- · 1. The guest browses the Rooms page and selects a room.
- · 2. The guest views room details (photos, amenities, rate, reviews).
- · 3. The guest clicks 'Book Now' and is directed to the Booking page.
- · 4. The guest selects check-in date, check-out date, pax count, and optional special requests.
- · 5. The system validates that check-out is after check-in.
- · 6. The system queries existing bookings for the room and checks for date overlaps.
- · 7. No conflicts found — the system creates the booking via Firestore runTransaction.
- · 8. The booking is saved with status 'Pending', calculated total cost, and payment initialized to zero.
- · 9. All FO staff receive notifications about the new booking request.
- · 10. The guest sees a success confirmation.

- Activity Diagram

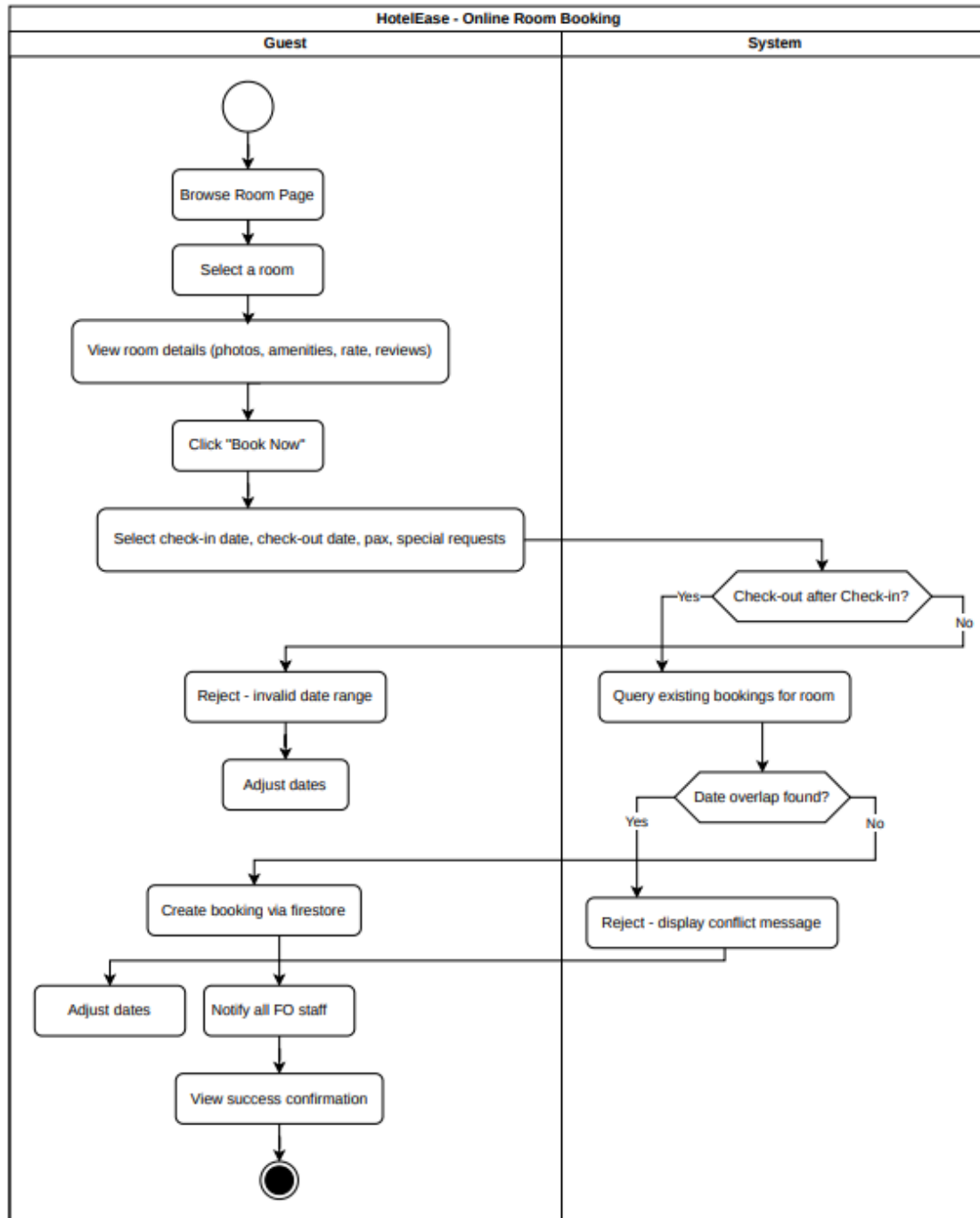


Figure 9. An Activity Diagram of Room & Booking Management

- Wireframe

HotelEaseBSHM-PMS

RoomsMy Bookings

 ProfileLogout

Booking

Select your dates, enter guest details, and confirm your reservation.

Room

Rapzz • 1

Rate: PHP 800 / night

Current status: Available

Check-in (2:00 PM)

mm/dd/yyyy

Check-out (12:00 NN)

mm/dd/yyyy

Number of Guests (pax)

1

Special Requests

e.g., late arrival, extra towels

Nights —

Total Cost PHP —

Confirm Booking (Pending)

Back to Rooms

Booking Calendar (Room)

Existing bookings for this room. Date conflicts are blocked on submit.

<>today

July 2026

Sun	Mon	Tue	Wed	Thu	Fri	Sat
28	29	30	1	2	3	4
				Checked Out	Cancelled	Pending
5	6	7	8	9	10	11
Checked Out						
Pending						
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	1
2	3	4	5	6	7	8

HotelEase · © 2026 All rights reserved.

AboutContactPrivacy

Figure 10: A Room Booking page of HotealEase

2.2 Transaction Name: Online Room Booking

- Use Case Diagram

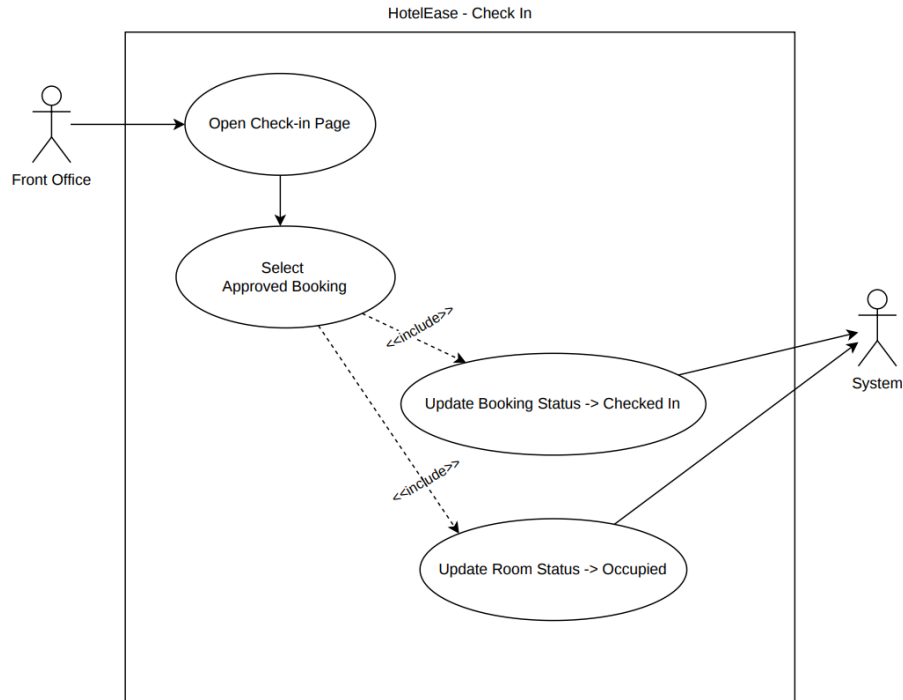


Figure 11: A Use Case Diagram for Check-In

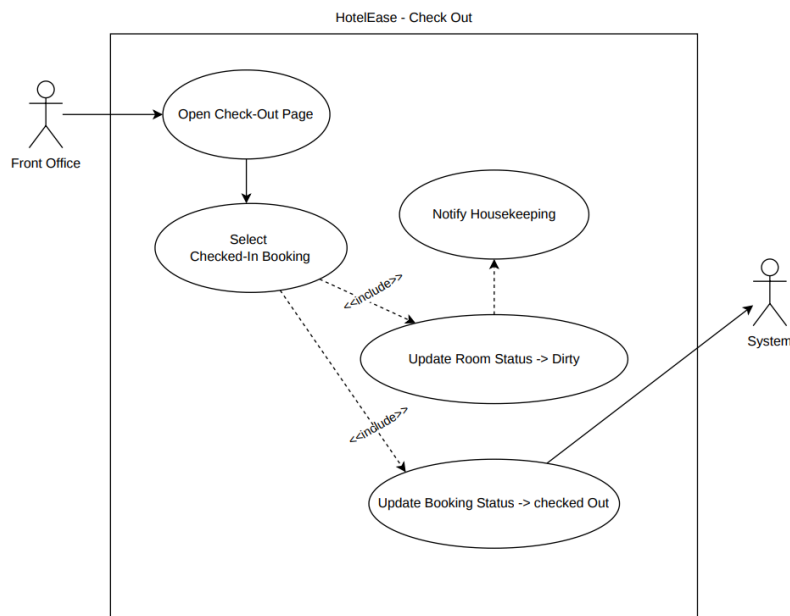


Figure 12: A Use Case Diagram for Check-Out

- Use Case Description

FO staff process guest check-in by transitioning an approved booking to 'Checked In' and the room to 'Occupied'. During check-out, the booking transitions to 'Checked Out' and the room status changes to 'Dirty / Needs Cleaning', triggering housekeeping notifications.

- · Preconditions: Check-in: Booking status must be 'Approved'. Check-out: Booking status must be 'Checked In'.
- · Postconditions: Check-in: Booking → Checked In, Room → Occupied. Check-out: Booking → Checked Out, Room → Dirty / Needs Cleaning. FO staff notified for housekeeping.

Main Success Scenario:

- · 1. FO staff opens the Check-In page and selects an approved booking.
- · 2. The system executes a Firestore transaction updating both the booking and room status atomically.
- · 3. The guest is checked in; room status changes to Occupied.
- · 4. During checkout, FO opens Check-Out page and selects the active booking.
- · 5. The system executes a checkout transaction: booking → Checked Out, room → Dirty.
- · 6. FO staff are notified that the room needs cleaning.

- Activity Diagram

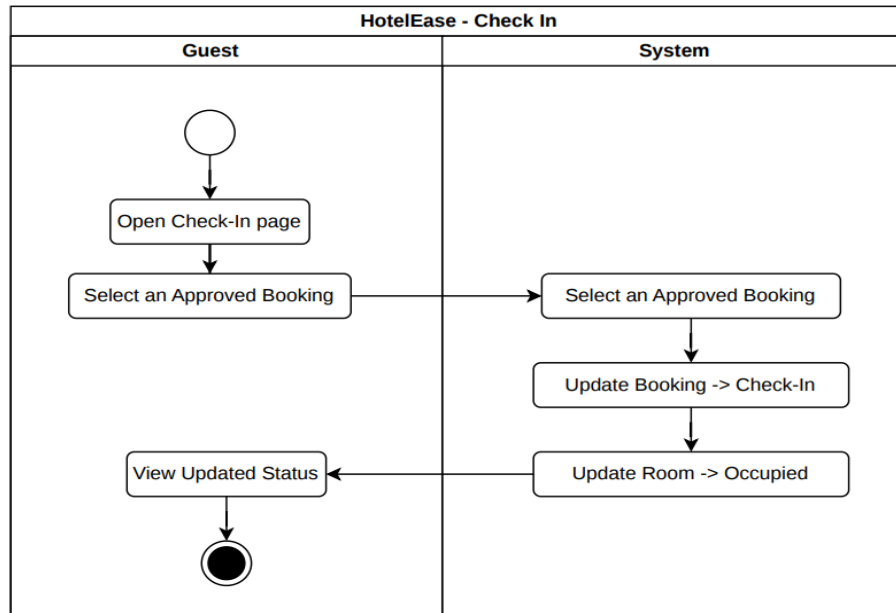


Figure 13: An Activity Diagram for Check-In

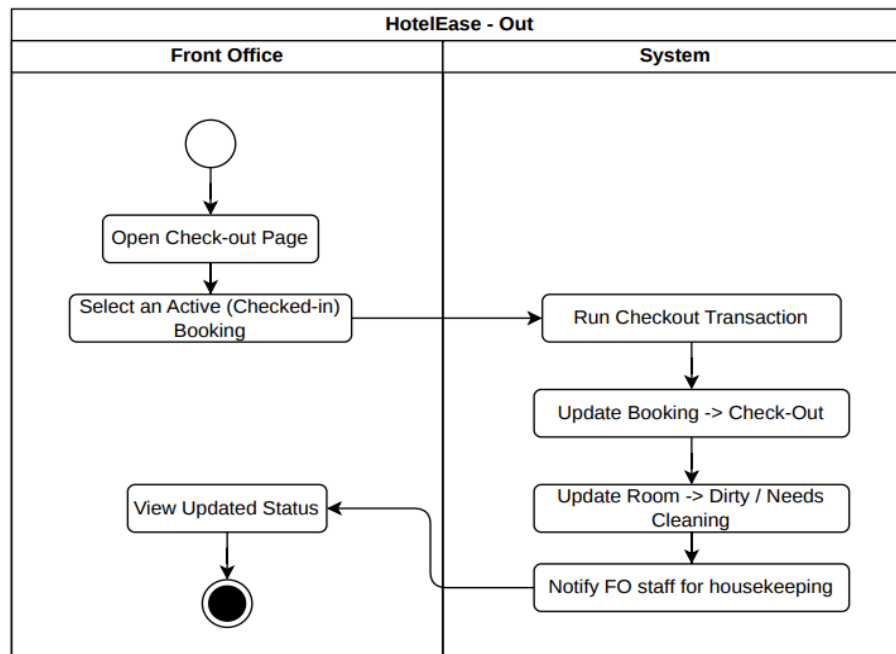


Figure 14: An Activity Diagram for Check-Out

- Wireframe

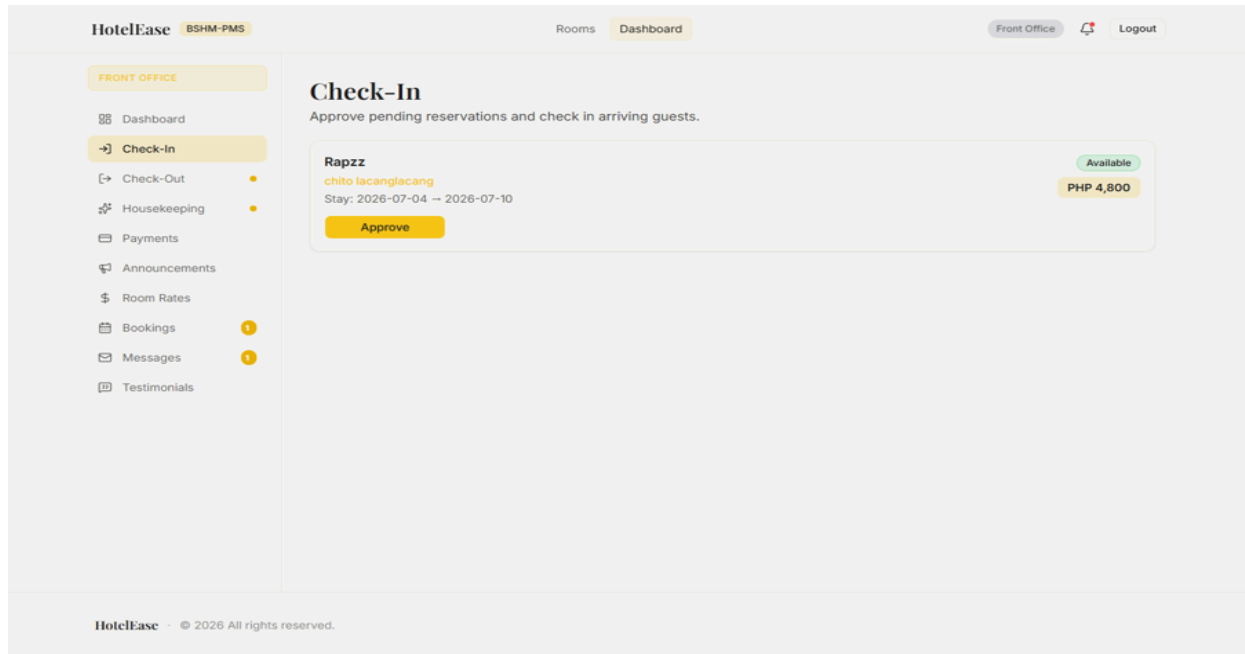


Figure 15: A Check-In Page of HotelEase

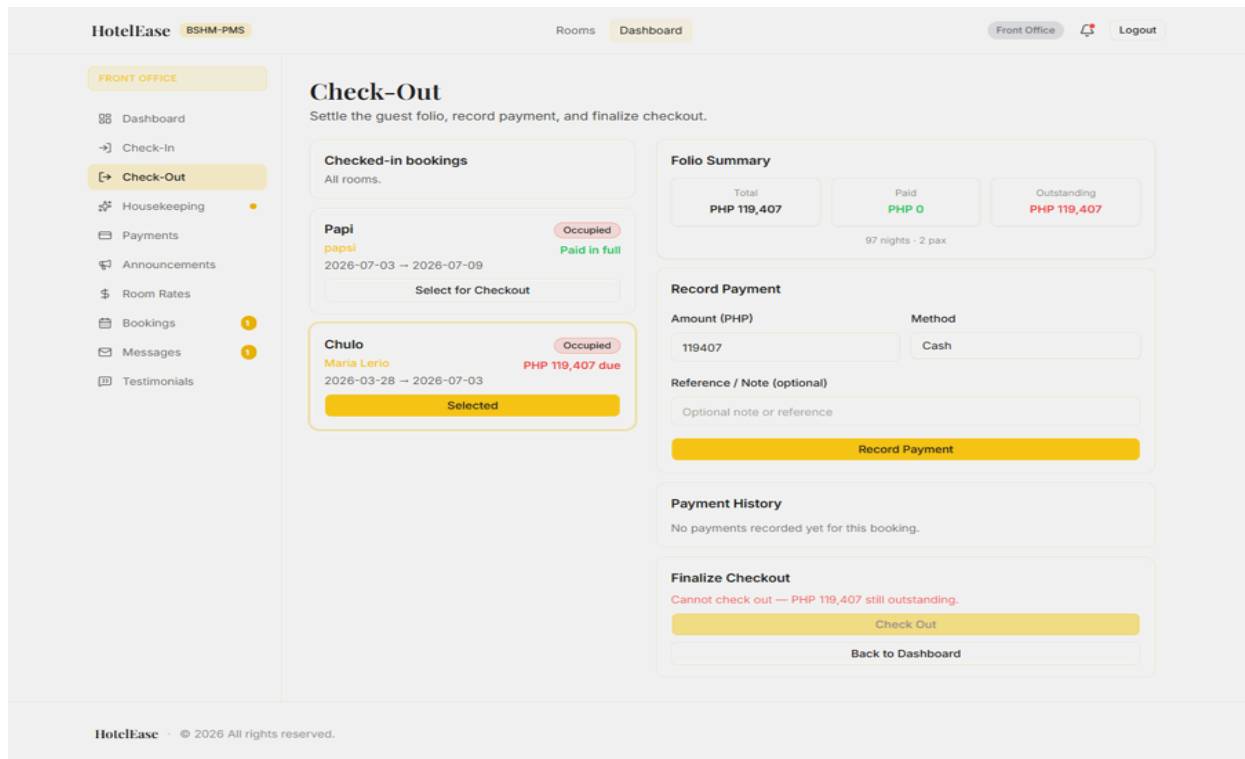


Figure 16: A Check-out Page for HotelEase

Module 3 – Housekeeping, Payments & Operations

3.1 Transaction Name: Housekeeping Workflow

- Use Case Diagram

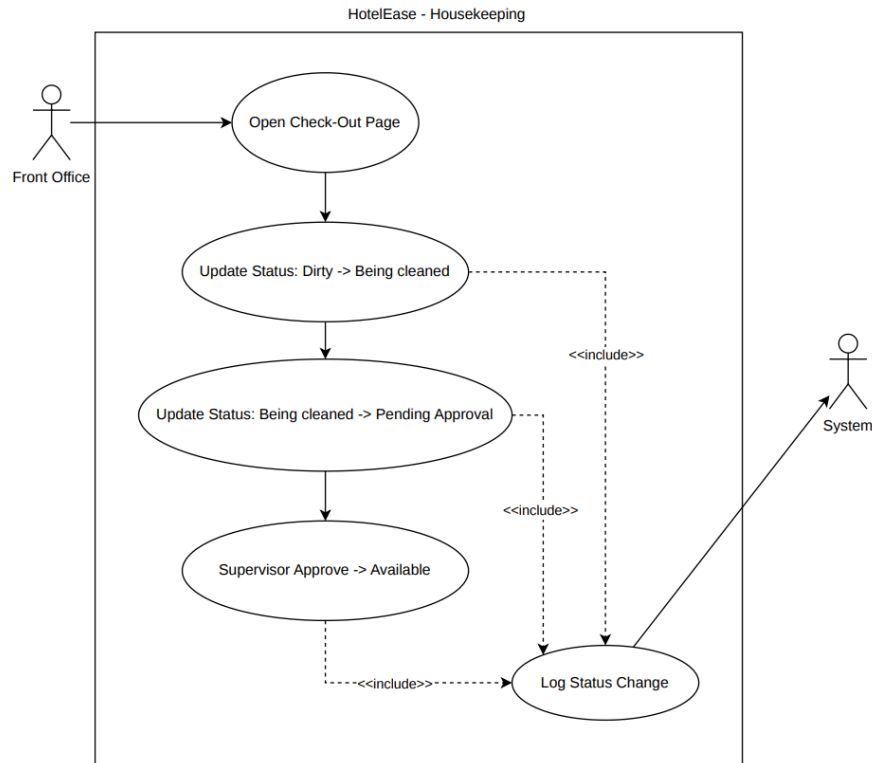


Figure 17: A Use Case Diagram For Housekeeping

- Use Case Description

When a guest checks out, the room status automatically transitions to 'Dirty / Needs Cleaning'. FO/Housekeeping staff update the status through the workflow: Dirty → Being Cleaned → Pending Approval → Available. Each status change is logged in the housekeeping_logs collection with timestamps and the role of the user who made the change.

- Preconditions: Room must exist in the system. User must have FO role.
- Postconditions: Room status is updated. A housekeeping log entry is created. FO staff are notified when rooms need cleaning.

Main Success Scenario:

- 1. FO staff opens the Housekeeping page.
- 2. Staff views rooms grouped by current status.
- 3. Staff selects a dirty room and updates status to 'Being Cleaned'.
- 4. After cleaning, staff updates status to 'Pending Approval'.
- 5. FO supervisor reviews and approves — room becomes 'Available'.
- 6. Each transition is logged with timestamp and role.

- Activity Diagram

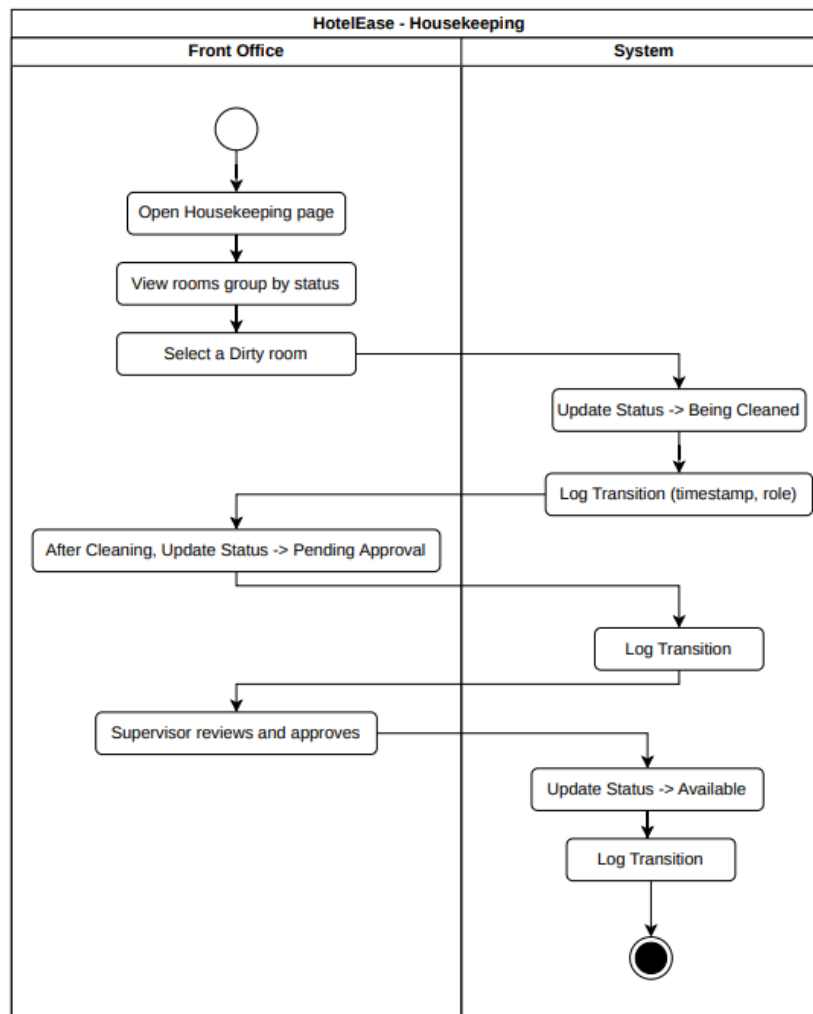


Figure 18: An Activity Diagram for Housekeeping

- Wireframe

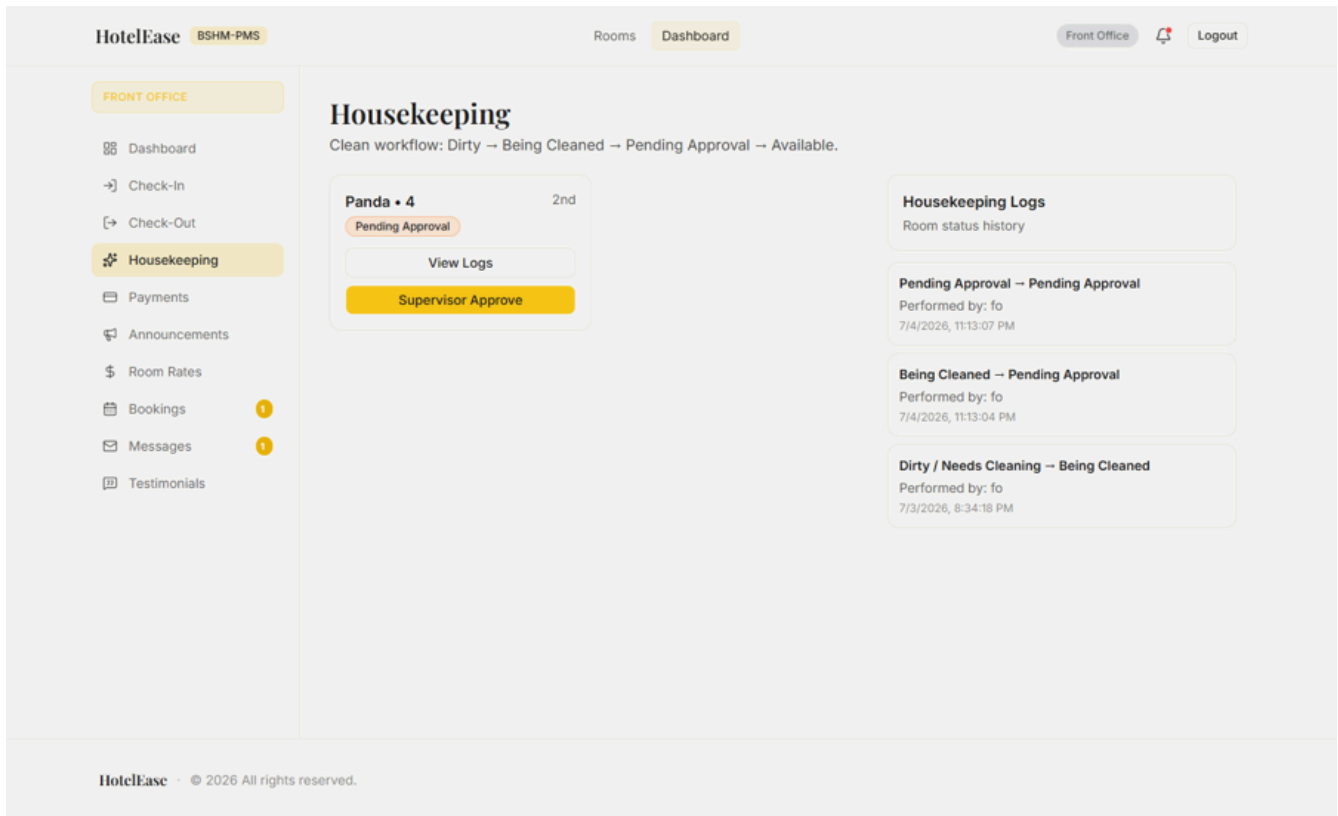


Figure 19: Housekeeping Page of HotelEase

3.2 Transaction Name : Record Payment

- Use Case Diagram

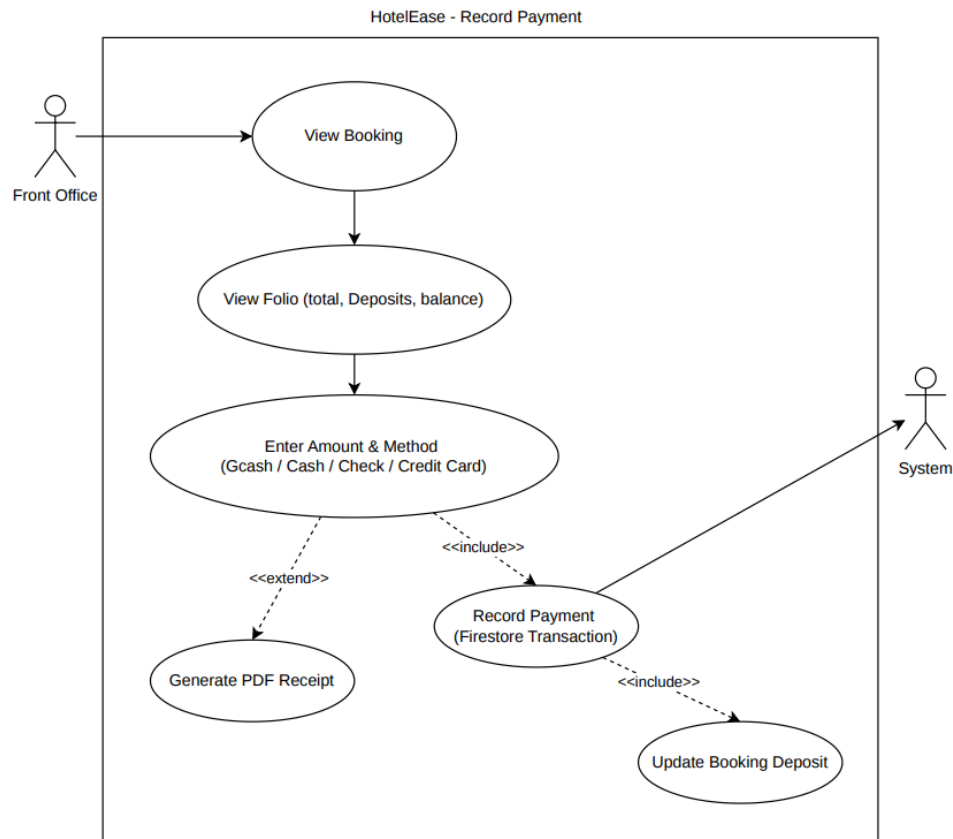


Figure 20: A Use Case Diagram for Payment

- Use Case Description

FO staff records a payment against a guest's booking. The system supports four payment methods: GCash (with reference number), Cash, Check (with check number and bank), and Credit Card (last 4 digits only). Payments are recorded atomically within a Firestore transaction that also updates the booking's deposit amount. Professional PDF receipts can be generated using jsPDF.

- Preconditions: A valid booking must exist. Payment amount must be positive. Payment method must be specified.
- Postconditions: A payment document is created. The booking's deposit is incremented. A PDF receipt can be generated.

Main Success Scenario:

- 1. FO staff opens the Payments page and selects a booking.
- 2. Staff views the folio: total cost, existing deposits, and outstanding balance.
- 3. Staff enters the payment amount and selects a payment method.
- 4. Staff provides method-specific details (reference number, check number, etc.).
- 5. The system executes a Firestore transaction: creates payment + updates booking deposit.
- 6. Staff optionally generates a PDF receipt for the guest.
- 7. The receipt is downloaded as a PDF file.

Activity Diagram

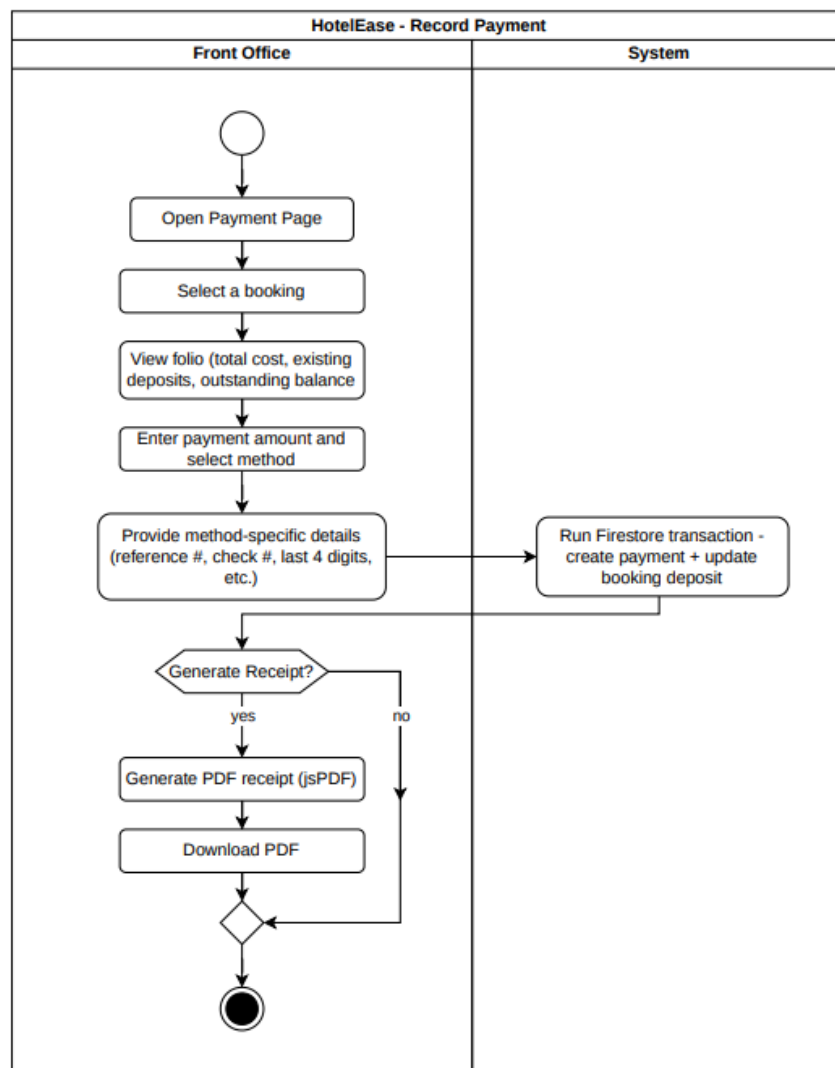


Figure 21: Activity Diagram for Payment

- Wireframe

HotelEaseBSHM-PMS

RoomsDashboard

Front OfficeLogout

FRONT OFFICE

Dashboard

Check-In

Check-Out

Housekeeping

Payments

Announcements

Room Rates

Bookings

Messages

Testimonials

Payments

Record and track payment transactions for active bookings.

Active Bookings

Select a booking to record payment.

Lofi Room

2026-07-06 → 2026-07-23

PHP 31,483 outstanding

Reserved

Folio Summary

Total

PHP 33,983

Paid

PHP 2,500

Outstanding

PHP 31,483

17 nights · 1 pax · Online

Record Payment

Amount (PHP)

Method

e.g. 2500

Cash

Reference / Note (optional)

Optional note or reference

Add Payment

Payment History

1 record

PHP 2,500

7/7/2026, 1:27:27 PM

Cash

Total recorded

PHP 2,500

HotelEase · © 2026 All rights reserved.

Figure 22: Payment Page of HotelEase

HotelEase

BSHM Property Management System

OFFICIAL RECEIPT

Receipt No: RCP-1783401716007

Date: 7/7/2026

Bill To:

Name: jay plarisan
Email: jay@gmail.com
Processed By: FO

Description	Details
Room	Minimal Room (Single Room)
Check-in	7/7/2026 at 2:00 PM
Check-out	7/15/2026 at 12:00 NN
Duration	8 night(s)
Rate per Night	Php 1,000.00

Summary	Amount
Subtotal	Php 8,000.00
Amount Paid	Php 7,999.00
Balance	Php 1.00
Payment Method	Credit Card

Thank you for choosing HotelEase!
This is a system-generated receipt.

Figure 23: A printable Receipt of HotelEase

3.3 Transaction Name : Announcements Management

- Use Case Diagram

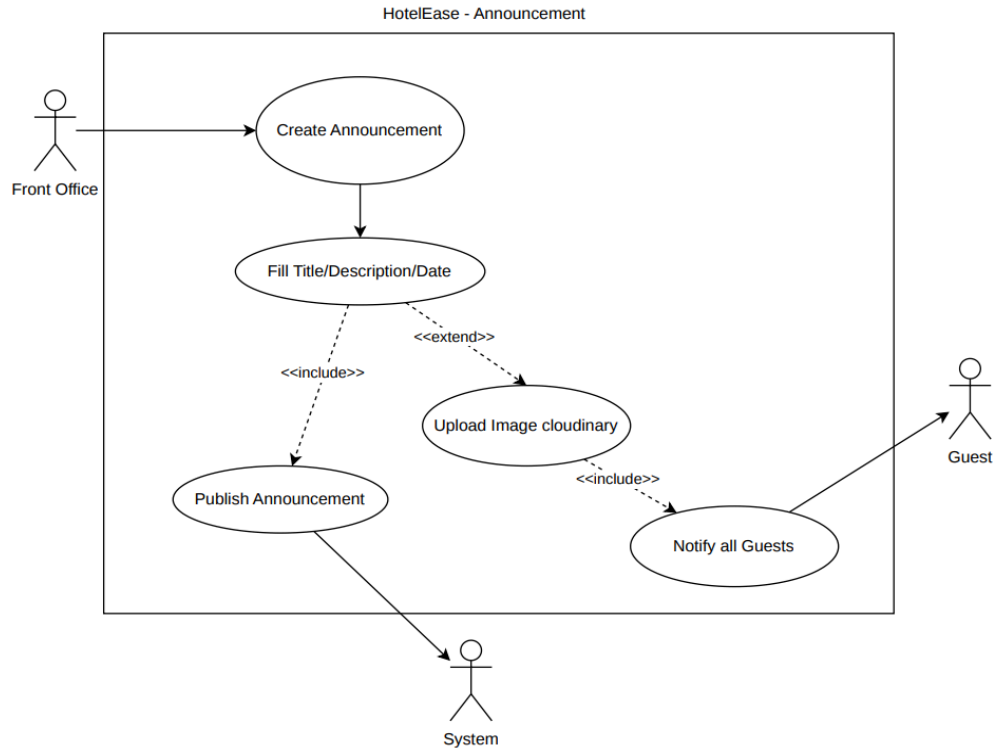


Figure 24: A Use Case Diagram for Announcement

- Use Case Description

FO staff creates event announcements with a title, description, date, and optional image (uploaded to Cloudinary). Announcements are displayed on the guest-facing landing page. All guest users receive in-app notifications when a new announcement is published.

- • Preconditions: User must have FO role. Title, description, and date are required.
- • Postconditions: Announcement is published. All guests receive notifications.

Main Success Scenario:

- • 1. FO staff opens the Announcements page.
- • 2. Staff clicks 'Create Announcement'.
- • 3. Staff fills in title, description, date, and optionally uploads an image.
- • 4. The system uploads the image to Cloudinary (if provided).
- • 5. The announcement is saved to Firestore with status 'Published'.
- • 6. All guest users receive in-app notifications.

- Activity Diagram

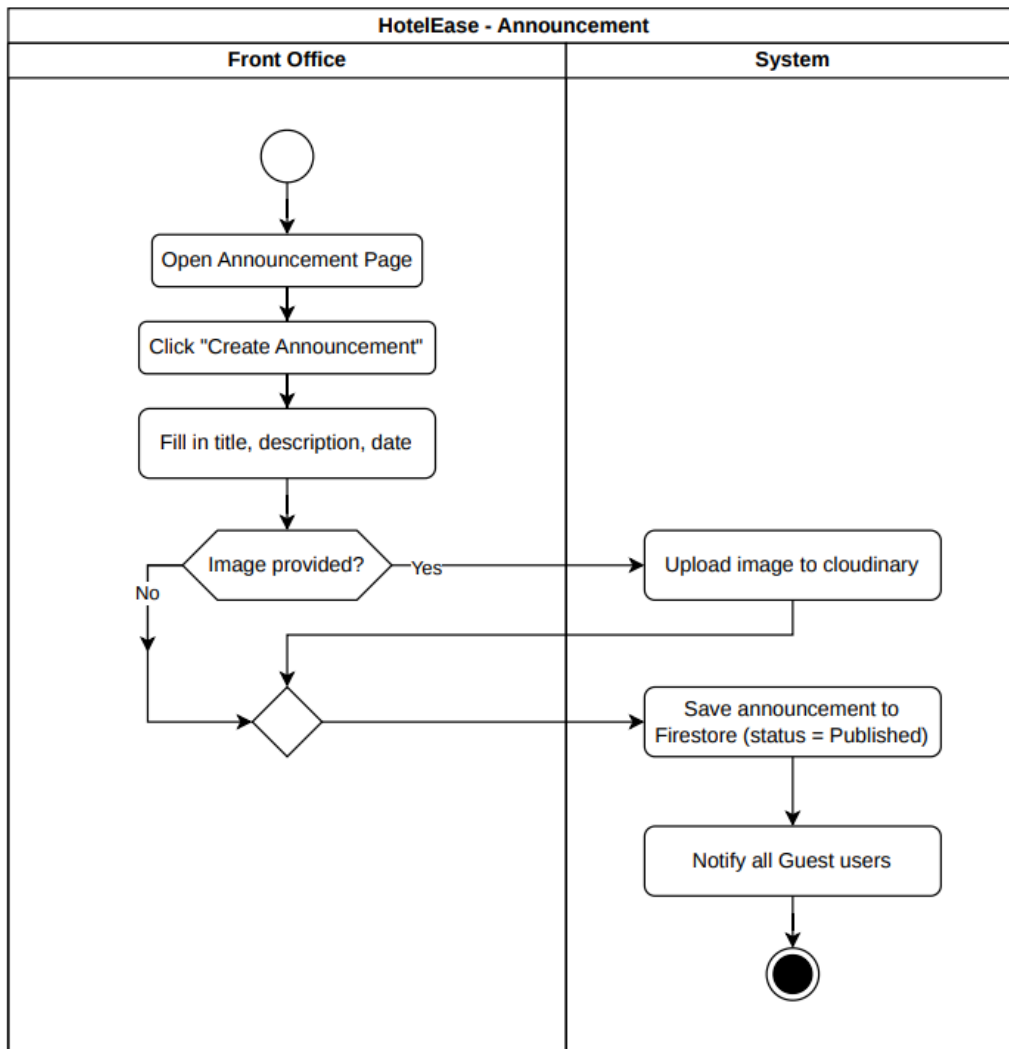


Figure 25: Activity Diagram for Announcement

- Wireframe

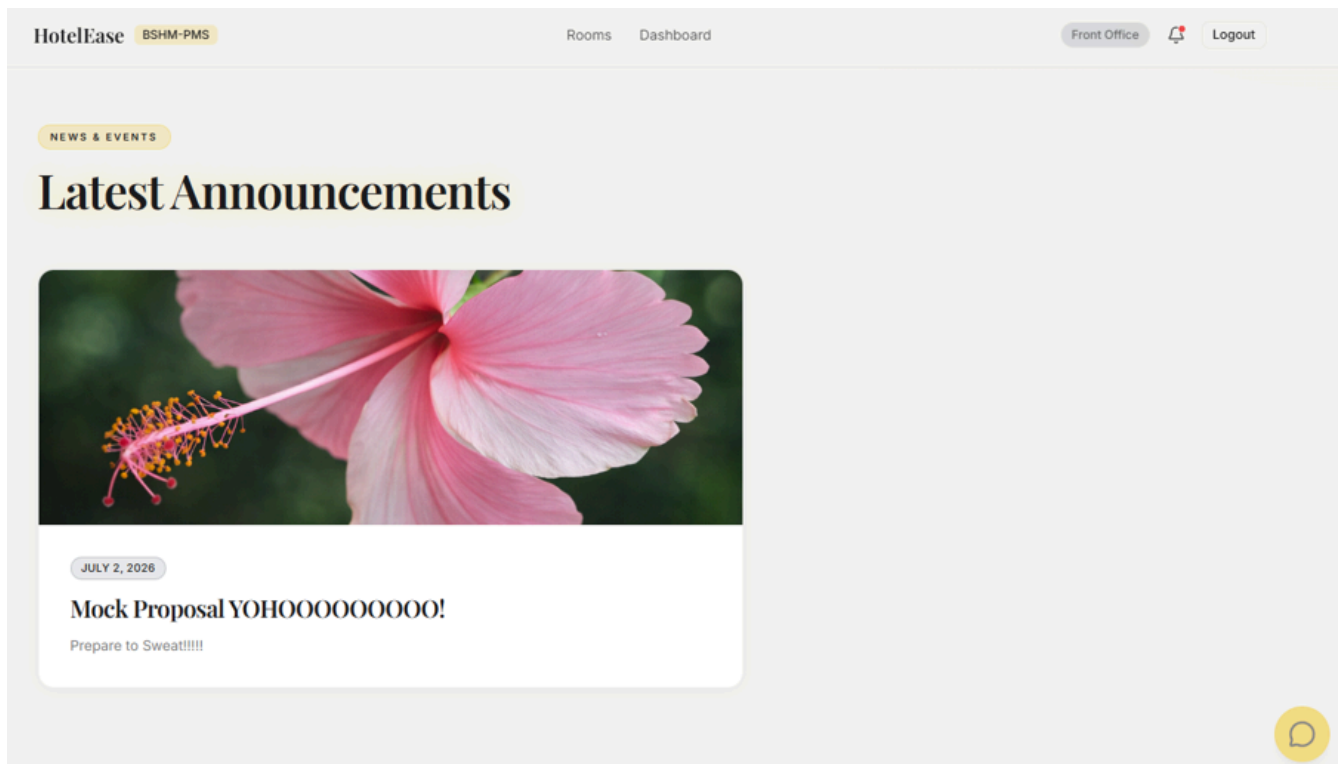


Figure 26: Announcement Section of HotelEase

3.4 Transaction Name : Guest Support Messaging

- Use Case Diagram

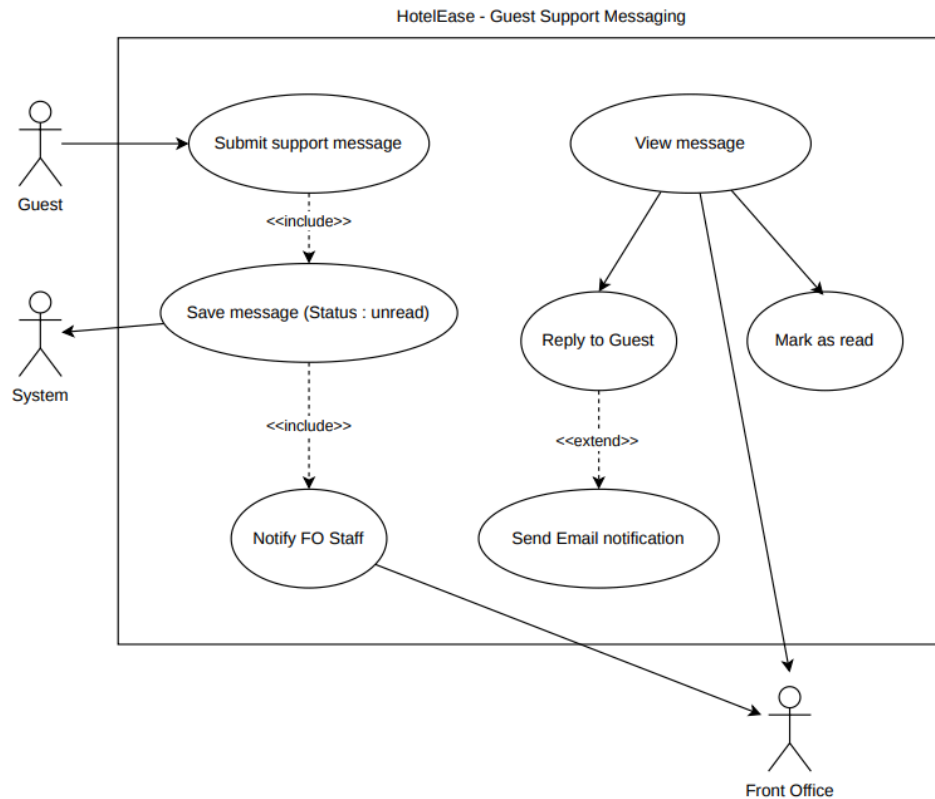


Figure 27: A Use Case Diagram for Support Messaging

- Use Case Description

Guests submit support messages via the Contact page with their name, email, subject, and message. FO staff receive in-app notifications and can view, mark as read, and reply to messages. Replies can optionally trigger email notifications via Firebase Cloud Functions.

- • Preconditions: All contact form fields must be completed.
- • Postconditions: Message is saved with status 'unread'. FO staff are notified.

Main Success Scenario:

- 1. Guest opens the Contact page.
- 2. Guest fills in name, email, subject, and message.
- 3. The system saves the message to Firestore with status 'unread'.
- 4. FO staff receive in-app notifications about the new message.
- 5. FO staff opens the Messages page and reads/replies to the message.

- Activity Diagram

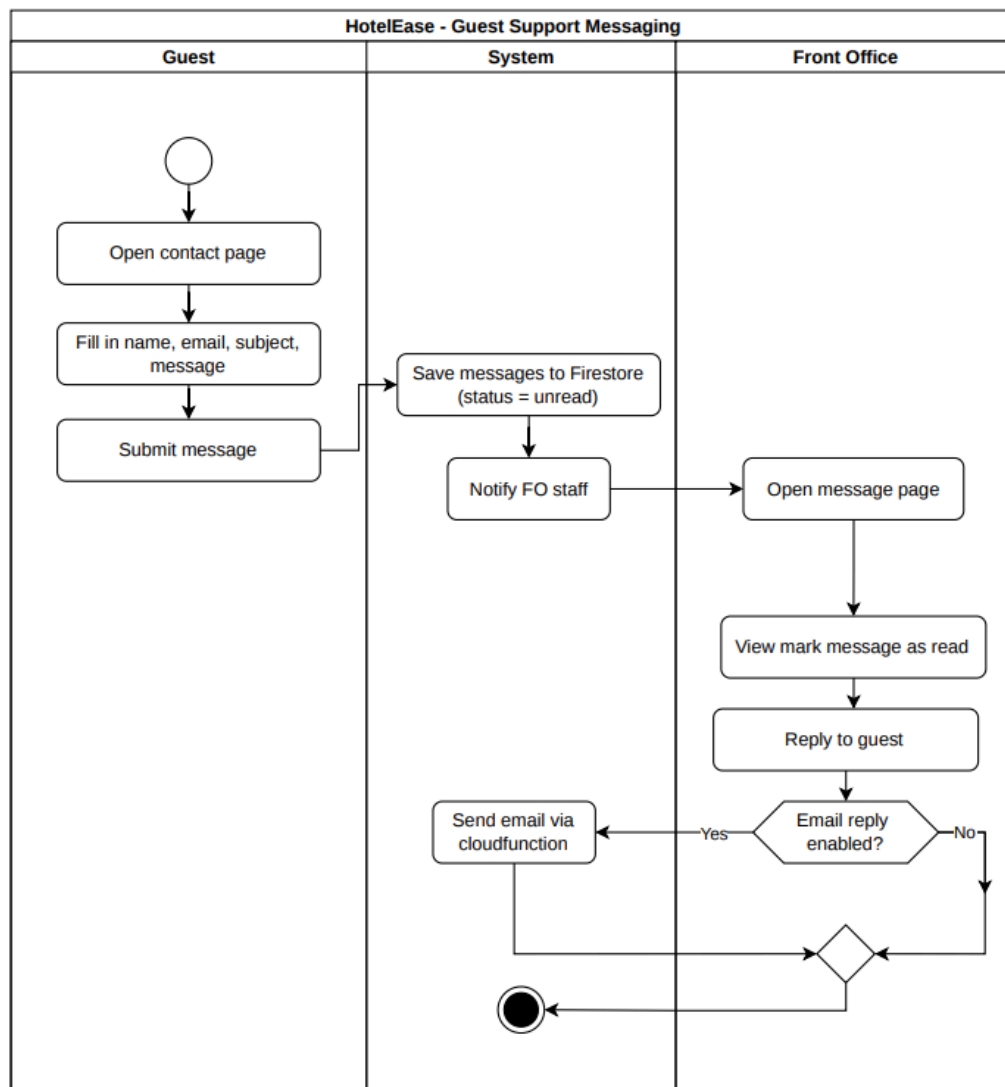


Figure 28: Activity Diagram for Support Messaging

- Wireframe

HotelEase BSHM-PMS Rooms Login Register

Contact Information

Reach out to the BSHM department support team.

LOCATION
BSHM Department, Consolatrix College of Toledo City

EMAIL
bshm-support@example.edu

PHONE
+63 000 000 0000

OFFICE HOURS
Mon-Fri, 8:00 AM - 5:00 PM

SOCIAL

Send a Message

We typically respond within office hours.

Full Name
Your full name

Email
you@example.com

Subject
General Inquiry

Message
Tell us how we can help...

Submit Message

HotelEase · © 2026 All rights reserved. About Contact Privacy

Figure 29: Contact Support Page of HotelEase

HotelEase BSHM-PMS Rooms Dashboard Front Office Logout

FRONT OFFICE

- Dashboard
- Check-In
- Check-Out
- Housekeeping
- Payments
- Announcements
- Room Rates
- Bookings 1
- Messages**
- Testimonials

Guest Messages

Review, respond, and track guest support inquiries.

All Unread 1 Replied

Name	Email	Subject	Status	Date	Actions
Rapzz	fo@gmail.com	General Inquiry	Unread	7/3/2026, 8:42:55 PM	View Mark as Read

Figure 30: Guest Messaging Page of HotelEase

Module 4 – Administration & Analytics

4.1 Transaction Name: Analytics Dashboard

- Use Case Diagram

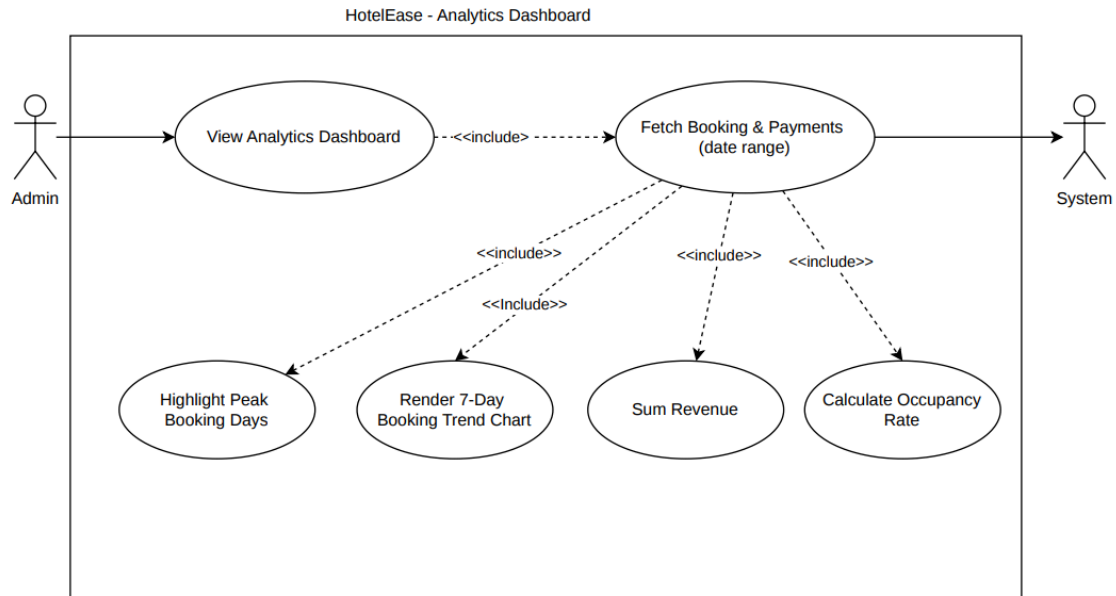


Figure 31: A Use Case Diagram for Admin and Analytics

- Use Case Description

The Admin accesses the Analytics Dashboard to view KPIs: occupancy rate (occupied nights / total available nights), total revenue from payments, booking trend chart (last 7 days), and peak booking days. Date range filtering is supported.

- Preconditions: User must have Admin role.
- Postconditions: Analytics data is displayed. No data is modified.

Main Success Scenario:

- 1. Admin navigates to the Analytics page.
- 2. System fetches bookings and payments within the date range.
- 3. Occupancy rate is calculated from booking nights vs. available room-nights.
- 4. Revenue is summed from all payments in the range.
- 5. A 7-day booking trend chart is displayed using Recharts.
- 6. Peak booking days (top 3) are highlighted.

- Activity Diagram

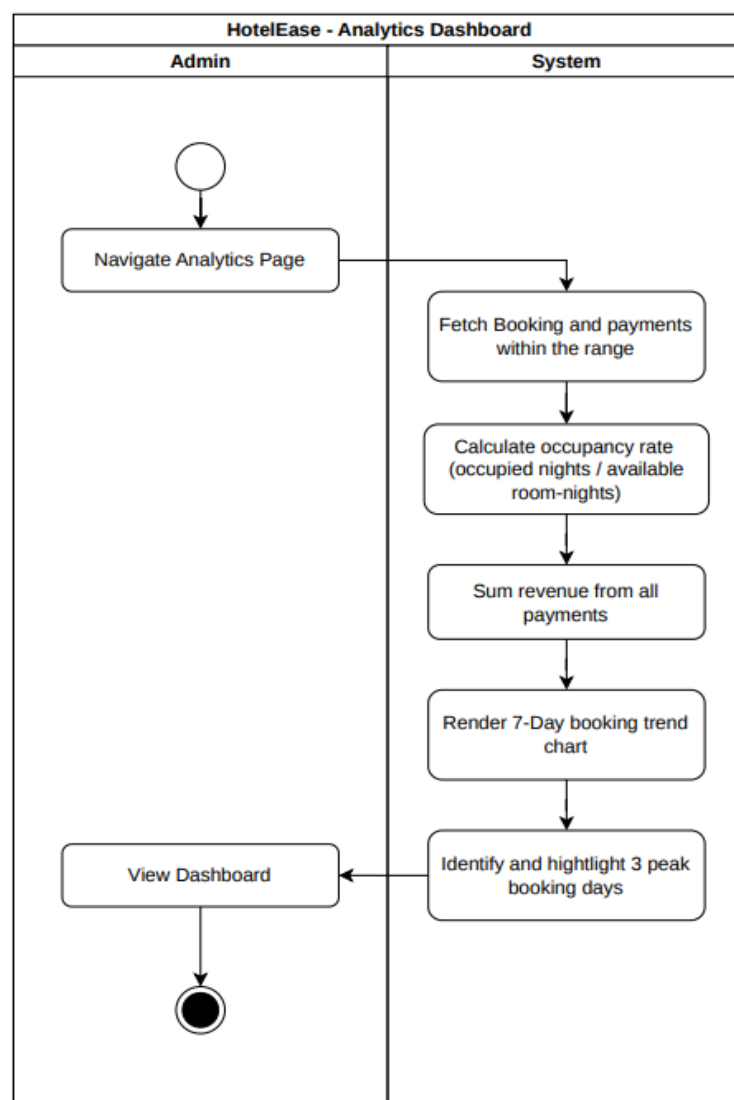


Figure 32: Activity Diagram for Admin and Analytics

- Wireframe

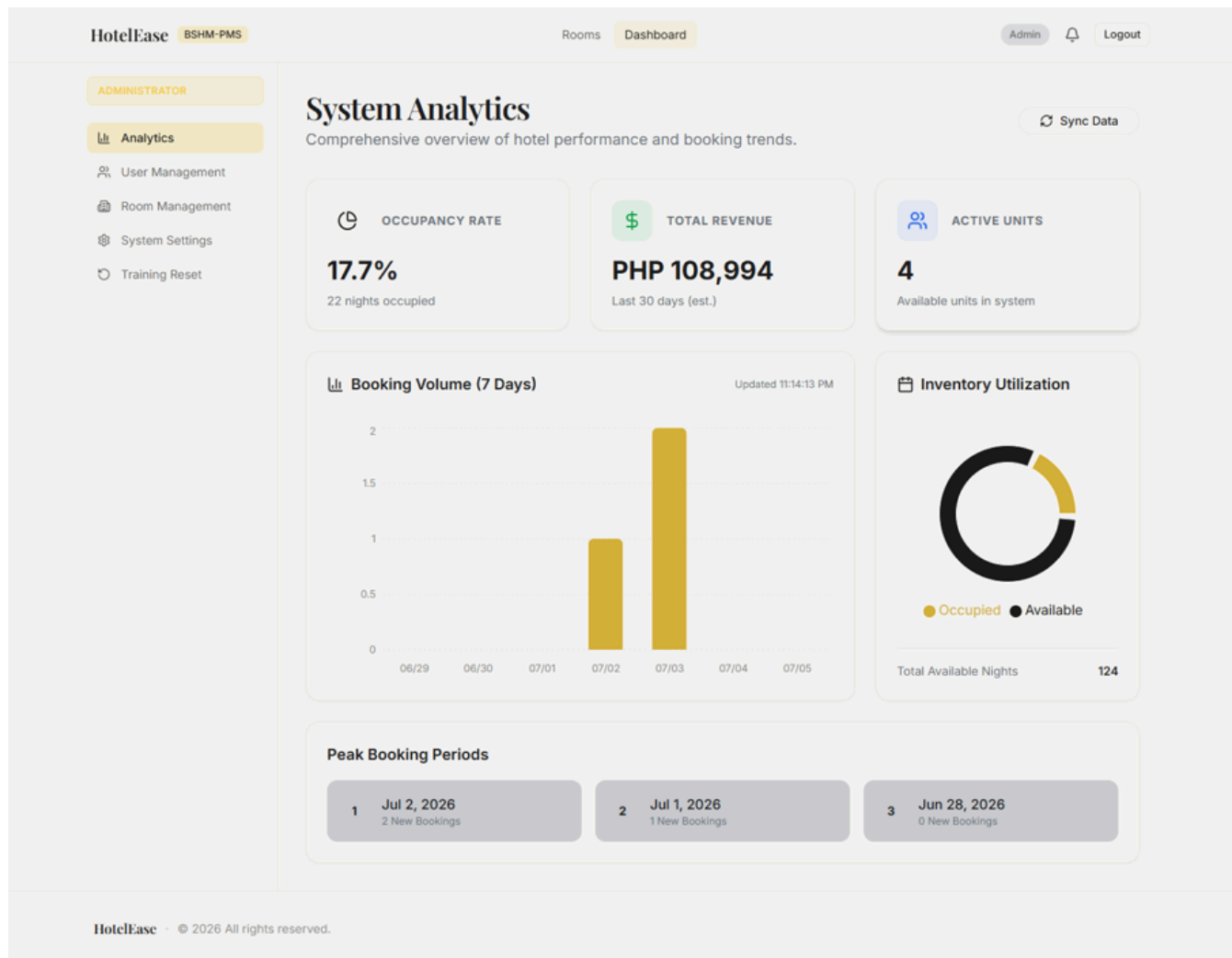


Figure 33: Analytics Dashboard of HotelEase

4.2 Transaction Name: Training Mode Management

- Use Case Diagram

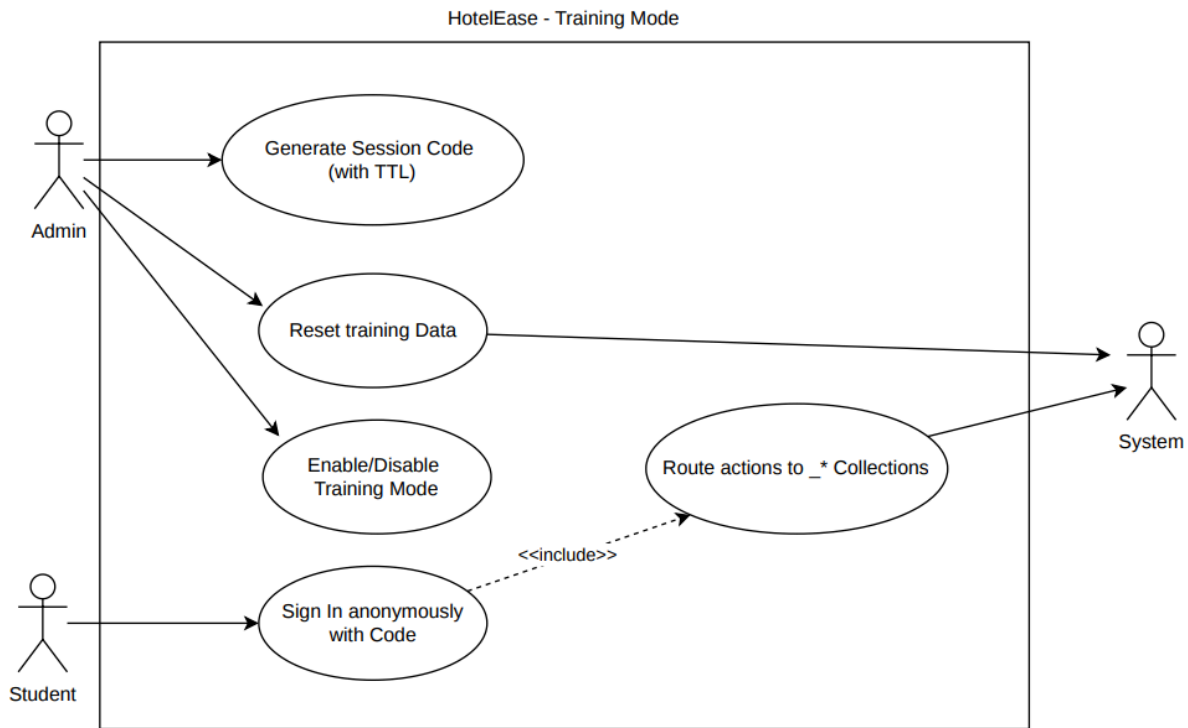


Figure 34: A Use Case Diagram for Training Management

- Use Case Description

Admin toggles Training Mode on/off. When enabled, all operations route to sandboxed collections (training_ prefix). Admin generates time-limited session codes for students. Training data can be reset between sessions.

- Preconditions: User must have Admin role.
- Postconditions: Training mode state is updated. Session codes generated with expiry. Training data can be cleared.

Main Success Scenario:

- 1. Admin opens System Settings and enables Training Mode.
- 2. Admin generates a training session code with a TTL (hours).
- 3. Students use the session code to sign in anonymously with a selected role.
- 4. All student actions write to training_ collections, isolated from production.
- 5. After the session, Admin resets training data to clear all sandboxed records.

• Activity Diagram

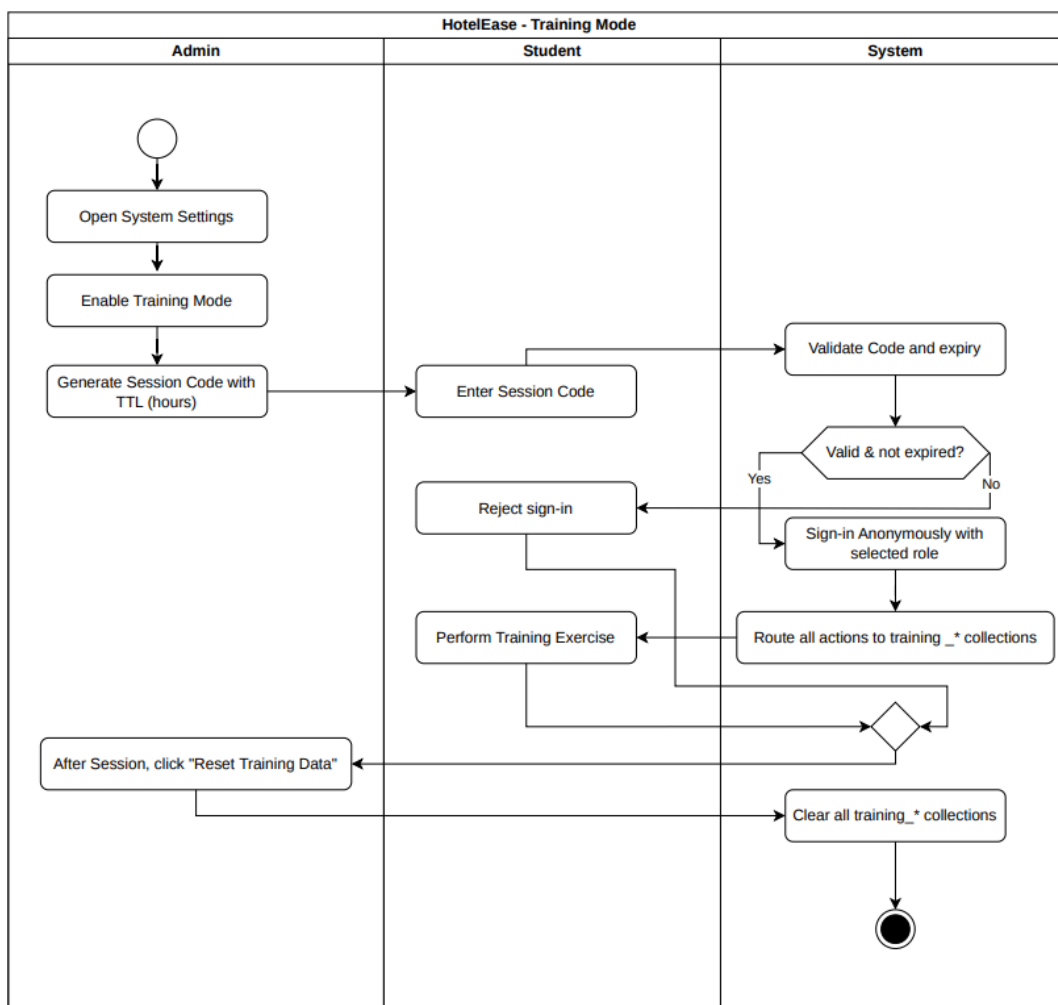


Figure 35: Activity Diagram for Training Management

● Wireframe

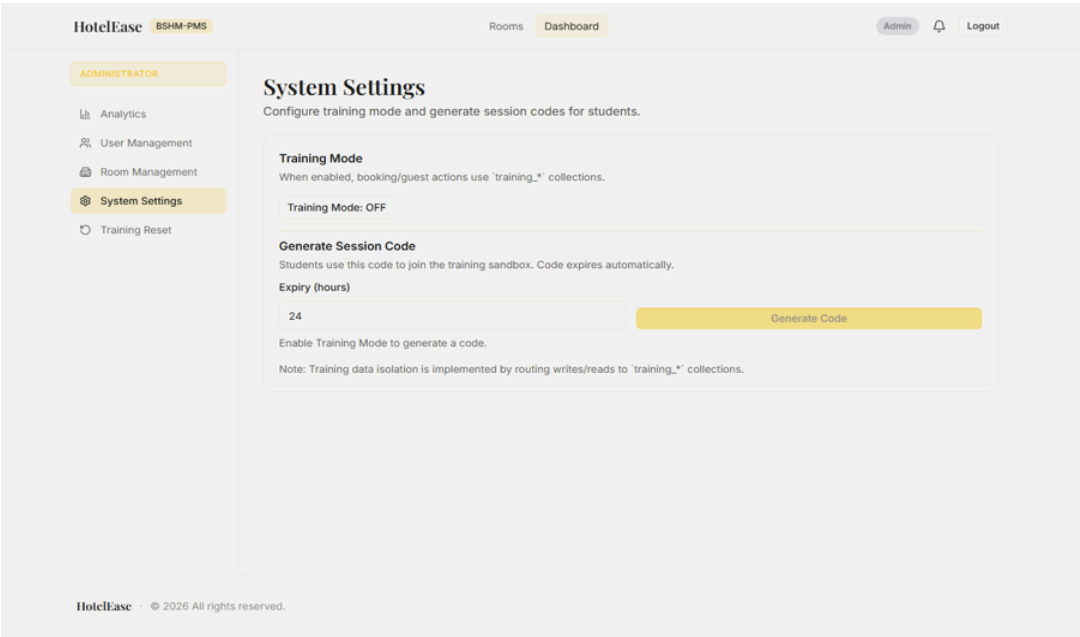


Figure 36: System Settings Page of HotelEase

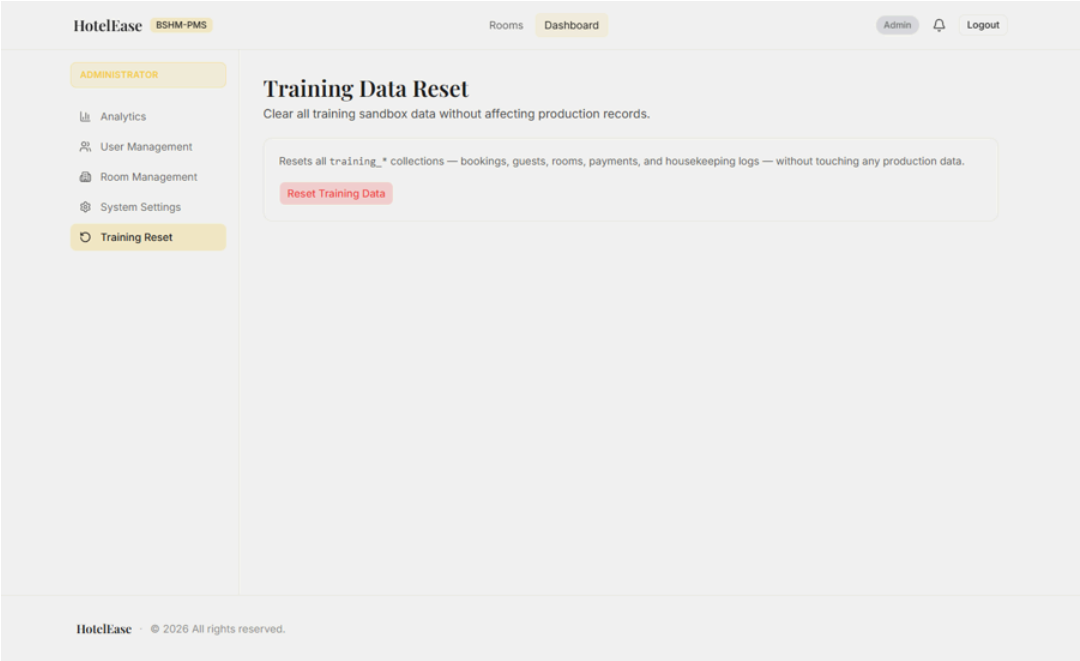


Figure 37: Training Data Reset Page of HotelEase

3.4 Non-functional requirements

Performance

HotelEase must meet the following performance criteria:

Response Time:

- Pages should load within 3 seconds on standard broadband connections, including Firestore data fetching and rendering.
- Booking operations (create, approve, check-in, check-out) should complete within 2 seconds, including Firestore transaction execution.
- Payment recording and receipt generation should complete within 3 seconds.

Concurrent Users:

- The system should support at least 100 concurrent users without significant performance degradation, leveraging Firestore's automatic scaling.
- Real-time listeners (onSnapshot) should deliver updates within 1 second of data changes in Firestore.

Database Performance:

- Firestore queries with proper indexing should return results within 500 milliseconds.
- The system should handle 1,000+ room documents, 10,000+ booking documents, and 50,000+ payment records without performance degradation.

File Upload:

- Room image uploads to Cloudinary should complete within 5 seconds for files up to 5MB each.
- Upload progress is tracked and displayed to the user via XMLHttpRequest progress events.

Security

Security measures for HotelEase:

User Authentication:

- · Firebase Authentication handles all credential management with industry-standard encryption.
- · Passwords are hashed using Firebase's built-in secure hashing (scrypt-based).
- · Session persistence is set to browserLocalPersistence to maintain authentication state across page reloads.
- · Anonymous authentication is used exclusively for training mode access with restricted permissions.

Data Encryption:

- · All communication uses HTTPS (TLS 1.2+) enforced by Firebase Hosting.
- · Firestore data is encrypted at rest using Google Cloud's default encryption (AES-256).
- · No sensitive payment data (full credit card numbers) is stored — only the last 4 digits are captured for reference.

Access Control:

- · Role-based access control (RBAC) is implemented at two layers: (1) Client-side via PrivateRoute component checking user role against allowedRoles, and (2) Server-side via Firestore Security Rules with helper functions (isProdFOOrAdmin, isAdmin, isOwner).
- · Anonymous users (training mode) are explicitly blocked from writing to production collections.
- · Guests can only create bookings where guestId matches their authenticated UID.

Input Validation:

- · All service functions validate input parameters (type checking, null guards, string trimming) before Firestore operations.
- · Firestore Security Rules enforce data integrity constraints (e.g., booking guestId must match auth UID).
- · Date validation prevents check-out before check-in and detects overlapping bookings.

Data Backup and Recovery:

- · Firestore provides automatic daily backups as part of Google Cloud's managed infrastructure.
- · Training data is isolated in separate collections and can be reset independently without affecting production data.

Reliability

Reliability requirements for continuous availability:

Uptime:

- The system targets 99.9% uptime by leveraging Firebase's globally distributed, managed infrastructure with automatic failover.

Failover Mechanism:

- Firebase services provide built-in failover across multiple Google Cloud data centers.
- Client-side Firestore queries include fallback mechanisms (e.g., `housekeepingService` retries queries without `orderBy` if a composite index is missing).

Redundancy:

- Firestore automatically replicates data across multiple zones within the selected region.
- Cloudinary CDN provides geographically distributed image delivery with built-in redundancy.

Error Handling:

- All service functions use try-catch blocks with descriptive error messages propagated to the UI via toast notifications (Sonner).
- Failed notification deliveries are caught and logged without blocking the primary operation.
- The AI chatbot returns a graceful fallback message ('I'm currently unavailable...') if the Groq API is unreachable or the API key is missing.
- Firestore `onSnapshot` listeners include error callbacks that fall back to one-time `getDocs` queries.

Database Consistency:

- Critical multi-document operations (booking creation, check-in, check-out, payment recording) use Firestore `runTransaction` to guarantee ACID properties.
- Room status and booking status are always updated atomically within the same transaction to prevent inconsistent states.